

Predicting Stars, Galaxies & Quasars with ML Model

Workflow of this notebook

- 1) [Introducing Dataset](#)
- 2) [Importing necessary libraries and modules for this notebook](#)
- 3) [Exploring the Dataset](#)
- 4) [Preparing data for our model](#)
- 5) [Scaling the data for our model and checking the distribution plots](#)
- 6) [Building the ML models and checking results](#)

About the Problem & Our Dataset -

So what exactly are stars, galaxies, and quasars?

- <p align="justify">A GALAXY is a gravitationally bound system of stars, stellar remnants, interstellar gas, dust, and dark matter. Galaxies are categorised according to their visual morphology as elliptical, spiral, or irregular. Many galaxies are thought to have supermassive black holes at their active centers.</p>
- <p align="justify">A STAR is a type of astronomical object consisting of a luminous spheroid of plasma held together by its own gravity. The nearest star to Earth is the Sun.
- <p align="justify">A QUASAR, also known as a quasi-stellar object, is an extremely luminous active galactic nucleus (AGN). The power radiated by quasars is enormous. A typical quasar is 27 trillion times brighter than our sun! If you were to place a quasar at the distance of Pluto, it would vaporise all of Earth's oceans to steam in a fifth of a second.

The dataset we'll be using for this model is the [Sloan Digital Sky Survey DR14](#)

The data consists of 10,000 observations of space taken by the SDSS. Every observation is described by 17 feature columns and 1 class column which identifies it to be either a star, galaxy or quasar. 30% of it is used in testing the model performance and 70% in training of the model.

Note: Upload the notebook to a kaggle session at the dataset link and run the cells

Importing necessary libraries and modules for the dataset

First of all we need to import all the packages we need. Numpy and Pandas for data manipulation and all the modules from sklearn for the machine learning feature

```
#Importing Necessary Libraries

'''Operating System Functionality'''
import os

'''Data Handling & Linear Algebra'''
import numpy as np
import pandas as pd

'''Visualisation'''
import matplotlib.pyplot as plt
import seaborn as sns

'''Manipulating Data and Model Building'''
import tensorflow as tf
from tensorflow import keras

'''Data Analysis'''
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

'''Measuring Classification Performance'''
from sklearn.metrics import accuracy_score

'''Classification & Regression'''
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC

'''Ignore warnings'''
```

```
import warnings
warnings.filterwarnings("ignore")

%matplotlib inline
```

Documents of the above used libraries and modules in case you aren't familiar and want to know more about it:-

- [os](#)
- [numpy](#)
- [pandas](#)
- [matplotlib.pyplot](#)
- [seaborn](#)
- [tensorflow](#)
- [keras](#)
- [All sklearn libraries](#)

Exploratory Analysis

```
#Loading data into a dataframe
data = pd.read_csv("/content/Skyserver_SQL2_27_2018_6_51_39_PM.csv")

#Displaying the first 5 rows of the dataset
data.head()

{"summary":{"\n  \"name\": \"data\",\n  \"rows\": 10000,\n  \"fields\": [\n    {\n      \"column\": \"objid\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 157703.88539140928,\n        \"min\": 1.23765e+18,\n        \"max\": 1.23765e+18,\n        \"num_unique_values\": 1,\n        \"samples\": [\n          1.23765e+18\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"ra\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 47.78343941253625,\n        \"min\": 8.235100497,\n        \"max\": 260.8843818,\n        \"num_unique_values\": 10000,\n        \"samples\": [\n          134.6492674\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"dec\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 25.21220657928434,\n        \"min\": -5.382632499,\n        \"max\": 68.54226541,\n        \"num_unique_values\": 10000,\n        \"samples\": [\n          54.38634445\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"u\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.8286560094113746,\n        \"min\": 12.98897,\n        \"max\": 19.5999,\n        \"num_unique_values\": 9730,\n        \"samples\": [\n
```

```

16.99432\n      ],\n      \"semantic_type\": \"\",\n      \"description\": \"\"\n    },\n    {\n      \"column\":\n      \"g\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.9454571752688135,\n        \"min\": 12.79955,\n        \"max\": 19.91897,\n        \"num_unique_values\": 9817,\n        \"samples\": [\n          17.88955\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      {\n        \"column\": \"r\",\n        \"properties\": {\n          \"dtype\": \"number\",\n          \"std\": 1.0677643494029374,\n          \"min\": 12.4316,\n          \"max\": 24.80204,\n          \"num_unique_values\": 9852,\n          \"samples\": [\n            17.62294\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n        },\n        {\n          \"column\": \"i\",\n          \"properties\": {\n            \"dtype\": \"number\",\n            \"std\": 1.1418048192775534,\n            \"min\": 11.94721,\n            \"max\": 28.17963,\n            \"num_unique_values\": 9890,\n            \"samples\": [\n              17.2252\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\"\n          },\n          {\n            \"column\": \"z\",\n            \"properties\": {\n              \"dtype\": \"number\",\n              \"std\": 1.2031880475759795,\n              \"min\": 11.61041,\n              \"max\": 22.83306,\n              \"num_unique_values\": 9896,\n              \"samples\": [\n                17.83397\n              ],\n              \"semantic_type\": \"\",\n              \"description\": \"\"\n            },\n            {\n              \"column\": \"run\",\n              \"properties\": {\n                \"dtype\": \"number\",\n                \"std\": 273,\n                \"min\": 308,\n                \"max\": 1412,\n                \"num_unique_values\": 23,\n                \"samples\": [\n                  1345\n                ],\n                \"semantic_type\": \"\",\n                \"description\": \"\"\n              },\n              {\n                \"column\": \"rerun\",\n                \"properties\": {\n                  \"dtype\": \"number\",\n                  \"std\": 0,\n                  \"min\": 301,\n                  \"max\": 301,\n                  \"num_unique_values\": 1,\n                  \"samples\": [\n                    301\n                  ],\n                  \"semantic_type\": \"\",\n                  \"description\": \"\"\n                },\n                {\n                  \"column\": \"camcol\",\n                  \"properties\": {\n                    \"dtype\": \"number\",\n                    \"std\": 1,\n                    \"min\": 1,\n                    \"max\": 6,\n                    \"num_unique_values\": 6,\n                    \"samples\": [\n                      4\n                    ],\n                    \"semantic_type\": \"\",\n                    \"description\": \"\"\n                  },\n                  {\n                    \"column\": \"field\",\n                    \"properties\": {\n                      \"dtype\": \"number\",\n                      \"std\": 162,\n                      \"min\": 11,\n                      \"max\": 768,\n                      \"num_unique_values\": 703,\n                      \"samples\": [\n                        621\n                      ],\n                      \"semantic_type\": \"\",\n                      \"description\": \"\"\n                    },\n                    {\n                      \"column\": \"specobjid\",\n                      \"properties\": {\n                        \"dtype\": \"number\",\n                        \"std\": 2.0139984928669066e+18,\n                        \"min\": 2.99578e+17,\n                        \"max\": 9.46883e+18,\n                        \"num_unique_values\": 6349,\n                        \"samples\": [\n                          3.11946e+17\n                        ],\n                        \"semantic_type\": \"\",\n                        \"description\": \"\"\n                      },\n                      {\n                        \"column\":

```



```

\"max\": 68.54226541,\n      \"num_unique_values\": 10000,\n\"samples\": [\n      54.38634445,\n      62.59739268,\n      -0.017859983\n    ],\n    \"semantic_type\": \"\",\n\"description\": \"\"\n  },\n  {\n    \"column\":\n    \"u\",\n    \"properties\": {\n      \"dtype\": \"number\",\n\"std\": 0.8286560094113746,\n      \"min\": 12.98897,\n\"max\": 19.5999,\n      \"num_unique_values\": 9730,\n\"samples\": [\n      16.99432,\n      18.27211,\n      19.51903\n    ],\n    \"semantic_type\": \"\",\n\"description\": \"\"\n  },\n  {\n    \"column\":\n    \"g\",\n    \"properties\": {\n      \"dtype\": \"number\",\n\"std\": 0.9454571752688135,\n      \"min\": 12.79955,\n\"max\": 19.91897,\n      \"num_unique_values\": 9817,\n\"samples\": [\n      17.88955,\n      18.89235,\n      15.92992\n    ],\n    \"semantic_type\": \"\",\n\"description\": \"\"\n  },\n  {\n    \"column\":\n    \"r\",\n    \"properties\": {\n      \"dtype\": \"number\",\n\"std\": 1.0677643494029374,\n      \"min\": 12.4316,\n\"max\": 24.80204,\n      \"num_unique_values\": 9852,\n\"samples\": [\n      17.62294,\n      15.54049,\n      18.64672\n    ],\n    \"semantic_type\": \"\",\n\"description\": \"\"\n  },\n  {\n    \"column\":\n    \"i\",\n    \"properties\": {\n      \"dtype\": \"number\",\n\"std\": 1.1418048192775534,\n      \"min\": 11.94721,\n\"max\": 28.17963,\n      \"num_unique_values\": 9890,\n\"samples\": [\n      17.2252,\n      18.65555,\n      16.67307\n    ],\n    \"semantic_type\": \"\",\n\"description\": \"\"\n  },\n  {\n    \"column\":\n    \"z\",\n    \"properties\": {\n      \"dtype\": \"number\",\n\"std\": 1.2031880475759795,\n      \"min\": 11.61041,\n\"max\": 22.83306,\n      \"num_unique_values\": 9896,\n\"samples\": [\n      17.83397,\n      16.71616,\n      17.0249\n    ],\n    \"semantic_type\": \"\",\n\"description\": \"\"\n  },\n  {\n    \"column\":\n    \"run\",\n    \"properties\": {\n      \"dtype\": \"number\",\n\"std\": 273,\n      \"min\": 308,\n      \"max\": 1412,\n\"num_unique_values\": 23,\n\"samples\": [\n      1345,\n      1332,\n      752\n    ],\n    \"semantic_type\": \"\",\n\"description\": \"\"\n  },\n  {\n    \"column\":\n    \"rerun\",\n    \"properties\": {\n      \"dtype\": \"number\",\n\"std\": 0,\n      \"min\": 301,\n      \"max\": 301,\n\"num_unique_values\": 1,\n\"samples\": [\n      301\n    ],\n    \"semantic_type\": \"\",\n    \"description\": \"\"\n  },\n  {\n    \"column\": \"camcol\",\n    \"properties\": {\n      \"dtype\": \"number\",\n      \"std\": 1,\n      \"min\": 1,\n      \"max\": 6,\n      \"num_unique_values\": 6,\n\"samples\": [\n      4\n    ],\n    \"semantic_type\":\n    \"\",\n    \"description\": \"\"\n  },\n  {\n    \"column\": \"field\",\n    \"properties\": {\n      \"dtype\":

```

```

\ "number\ ",\n          \ "std\ ": 162,\n          \ "min\ ": 11,\n
\ "max\ ": 768,\n          \ "num_unique_values\ ": 703,\n
\ "samples\ ": [\n          621\n          ],\n          \ "semantic_type\ ":
\ "\",\n          \ "description\ ": \ "\",\n          }\n          },\n          {\n
\ "column\ ": \ "class\ ",\n          \ "properties\ ": {\n          \ "dtype\ ":
\ "category\ ",\n          \ "num_unique_values\ ": 3,\n          \ "samples\ ":
[\n          \ "STAR\ "\n          ],\n          \ "semantic_type\ ": \ "\",\n
\ "description\ ": \ "\",\n          }\n          },\n          {\n          \ "column\ ":
\ "redshift\ ",\n          \ "properties\ ": {\n          \ "dtype\ ":
\ "number\ ",\n          \ "std\ ": 0.38877403581146536,\n          \ "min\ ": -
0.004136078,\n          \ "max\ ": 5.353854,\n
\ "num_unique_values\ ": 9637,\n          \ "samples\ ": [\n
0.000189883\n          ],\n          \ "semantic_type\ ": \ "\",\n
\ "description\ ": \ "\",\n          }\n          },\n          {\n          \ "column\ ":
\ "plate\ ",\n          \ "properties\ ": {\n          \ "dtype\ ": \ "number\ ",\n
\ "std\ ": 1788,\n          \ "min\ ": 266,\n          \ "max\ ": 8410,\n
\ "num_unique_values\ ": 487,\n          \ "samples\ ": [\n          3848\n
],\n          \ "semantic_type\ ": \ "\",\n          \ "description\ ": \ "\",\n
}\n          },\n          {\n          \ "column\ ": \ "mjd\ ",\n          \ "properties\ ": {\n
\ "dtype\ ": \ "number\ ",\n          \ "std\ ": 1511,\n
\ "min\ ": 51578,\n          \ "max\ ": 57481,\n
\ "num_unique_values\ ": 355,\n          \ "samples\ ": [\n          54862\n
],\n          \ "semantic_type\ ": \ "\",\n          \ "description\ ": \ "\",\n
}\n          },\n          {\n          \ "column\ ": \ "fiberid\ ",\n
\ "properties\ ": {\n          \ "dtype\ ": \ "number\ ",\n          \ "std\ ":
206,\n          \ "min\ ": 1,\n          \ "max\ ": 1000,\n
\ "num_unique_values\ ": 892,\n          \ "samples\ ": [\n          658\n
],\n          \ "semantic_type\ ": \ "\",\n          \ "description\ ": \ "\",\n
}\n          }\n          ]\n          }", "type": "dataframe", "variable_name": "data"}

```

data.shape

(10000, 16)

data.describe()

```

{"summary": "{\n  \ "name\ ": \ "data\ ",\n  \ "rows\ ": 8,\n  \ "fields\ ": [\n
n    {\n      \ "column\ ": \ "ra\ ",\n      \ "properties\ ": {\n
\ "dtype\ ": \ "number\ ",\n      \ "std\ ": 3484.388142825381,\n
\ "min\ ": 8.235100497,\n      \ "max\ ": 10000.0,\n
\ "num_unique_values\ ": 8,\n      \ "samples\ ": [\n
175.5299865838047,\n      180.39451394999998,\n      10000.0\n
],\n      \ "semantic_type\ ": \ "\",\n      \ "description\ ": \ "\",\n
}\n    },\n    {\n      \ "column\ ": \ "dec\ ",\n      \ "properties\ ": {\n
\ "dtype\ ": \ "number\ ",\n      \ "std\ ": 3528.610457701967,\n
\ "min\ ": -5.382632499,\n      \ "max\ ": 10000.0,\n
\ "num_unique_values\ ": 8,\n      \ "samples\ ": [\n
14.8361479841938,\n      0.40416602999999995,\n      10000.0\n
],\n      \ "semantic_type\ ": \ "\",\n      \ "description\ ": \ "\",\n
}\n    },\n    {\n      \ "column\ ": \ "u\ ",\n      \ "properties\ ": {\n

```

```

\"dtype\": \"number\",\\n      \"std\": 3530.068232364262,\\n
\"min\": 0.8286560094113746,\\n      \"max\": 10000.0,\\n
\"num_unique_values\": 8,\\n      \"samples\": [\\n
18.619355357999996,\\n      18.853095,\\n      10000.0\\n
n      ],\\n      \"semantic_type\": \"\",\\n
\"description\": \"\"\\n      }\\n      },\\n      {\\n      \"column\":
\"g\",\\n      \"properties\": {\\n      \"dtype\": \"number\",\\n
\"std\": 3530.318691147182,\\n      \"min\": 0.9454571752688135,\\n
\"max\": 10000.0,\\n      \"num_unique_values\": 8,\\n
\"samples\": [\\n      17.371931487,\\n
17.495134999999998,\\n      10000.0\\n      ],\\n
\"semantic_type\": \"\",\\n      \"description\": \"\"\\n      }\\n
n      },\\n      {\\n      \"column\": \"r\",\\n      \"properties\": {\\n
\"dtype\": \"number\",\\n      \"std\": 3530.2021967208884,\\n
\"min\": 1.0677643494029374,\\n      \"max\": 10000.0,\\n
\"num_unique_values\": 8,\\n      \"samples\": [\\n
16.840963317,\\n      16.85877,\\n      10000.0\\n      ],\\n
\"semantic_type\": \"\",\\n      \"description\": \"\"\\n      }\\n
n      },\\n      {\\n      \"column\": \"i\",\\n      \"properties\": {\\n
\"dtype\": \"number\",\\n      \"std\": 3530.1111414447496,\\n
\"min\": 1.1418048192775534,\\n      \"max\": 10000.0,\\n
\"num_unique_values\": 8,\\n      \"samples\": [\\n
16.583579249000003,\\n      16.554985000000002,\\n      10000.0\\n
n      ],\\n      \"semantic_type\": \"\",\\n
\"description\": \"\"\\n      }\\n      },\\n      {\\n      \"column\":
\"z\",\\n      \"properties\": {\\n      \"dtype\": \"number\",\\n
\"std\": 3530.4270302395526,\\n      \"min\": 1.2031880475759795,\\n
\"max\": 10000.0,\\n      \"num_unique_values\": 8,\\n
\"samples\": [\\n      16.42283316,\\n      16.389944999999997,\\n
10000.0\\n      ],\\n      \"semantic_type\": \"\",\\n
\"description\": \"\"\\n      }\\n      },\\n      {\\n      \"column\":
\"run\",\\n      \"properties\": {\\n      \"dtype\": \"number\",\\n
\"std\": 3268.3901534511706,\\n      \"min\": 273.3050240163418,\\n
\"max\": 10000.0,\\n      \"num_unique_values\": 8,\\n
\"samples\": [\\n      981.0348,\\n      756.0,\\n
10000.0\\n      ],\\n      \"semantic_type\": \"\",\\n
\"description\": \"\"\\n      }\\n      },\\n      {\\n      \"column\":
\"rerun\",\\n      \"properties\": {\\n      \"dtype\": \"number\",\\n
\"std\": 3445.927233706481,\\n      \"min\": 0.0,\\n      \"max\":
10000.0,\\n      \"num_unique_values\": 3,\\n      \"samples\": [\\n
10000.0,\\n      301.0,\\n      0.0\\n      ],\\n
\"semantic_type\": \"\",\\n      \"description\": \"\"\\n      }\\n
n      },\\n      {\\n      \"column\": \"camcol\",\\n      \"properties\":
{\\n      \"dtype\": \"number\",\\n      \"std\":
3534.356739485042,\\n      \"min\": 1.0,\\n      \"max\": 10000.0,\\n
\"num_unique_values\": 8,\\n      \"samples\": [\\n      3.6487,\\n
4.0,\\n      10000.0\\n      ],\\n      \"semantic_type\": \"\",\\n
n      \"description\": \"\"\\n      }\\n      },\\n      {\\n
\"column\": \"field\",\\n      \"properties\": {\\n      \"dtype\":

```

```

{"number": 11.0, "std": 3434.636724924114, "min": 11.0, "max": 10000.0, "num_unique_values": 8, "samples": [302.3801, 299.0, 10000.0], "semantic_type": "", "description": "", "column": "redshift", "properties": {"dtype": "number", "std": 3535.230456223175, "min": 0.004136078, "max": 10000.0, "num_unique_values": 8, "samples": [0.14372571182796998, 0.042590705, 10000.0], "semantic_type": "", "description": "", "column": "plate", "properties": {"dtype": "number", "std": 3843.6946645592875, "min": 266.0, "max": 10000.0, "num_unique_values": 8, "samples": [1460.9864, 441.0, 10000.0], "semantic_type": "", "description": "", "column": "mjd", "properties": {"dtype": "number", "std": 22251.116071451008, "min": 1511.1506506367791, "max": 57481.0, "num_unique_values": 8, "samples": [52943.5333, 51997.0, 10000.0], "semantic_type": "", "description": "", "column": "fiberid", "properties": {"dtype": "number", "std": 3416.657012243322, "min": 1.0, "max": 10000.0, "num_unique_values": 8, "samples": [353.0694, 351.0, 10000.0], "semantic_type": "", "description": ""}}
n }n ]n}", "type": "dataframe"}

```

#Checking for null values to determine completeness of the dataset
data.info()

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 16 columns):
#   Column      Non-Null Count  Dtype
---  -
0    ra          10000 non-null  float64
1    dec         10000 non-null  float64
2    u           10000 non-null  float64
3    g           10000 non-null  float64
4    r           10000 non-null  float64
5    i           10000 non-null  float64
6    z           10000 non-null  float64
7    run         10000 non-null  int64
8    rerun       10000 non-null  int64
9    camcol      10000 non-null  int64
10   field       10000 non-null  int64

```

```

11 class      10000 non-null object
12 redshift   10000 non-null float64
13 plate      10000 non-null int64
14 mjd        10000 non-null int64
15 fiberid    10000 non-null int64
dtypes: float64(8), int64(7), object(1)
memory usage: 1.2+ MB

```

No missing data so the dataset is complete with no voids and missing cells

The Target from data is Data classification to Star Galaxy or Quasar, so the class column has 3 Categories and in this case we need to convert them into numeric data.

Sklearn provides a very efficient tool for encoding the levels of categorical features into numeric values. `LabelEncoder().fit` encodes labels with a value between 0 and `n_classes-1` where `n` is the number of distinct labels. If a label repeats it assigns the same value to as assigned earlier. You can read more about it [here!](#)

```

le = LabelEncoder().fit(data['class'])
data['class'] = le.transform(data['class'])

```

The Final Dataset

#Printing the dataset after all the changes to check the dataset
`data.head(10)`

```

{"summary":{"\n  \"name\": \"data\", \n  \"rows\": 10000, \n  \"fields\": [\n    {\n      \"column\": \"ra\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 47.78343941253625, \n        \"min\": 8.235100497, \n        \"max\": 260.8843818, \n        \"num_unique_values\": 10000, \n        \"samples\": [\n          134.6492674, \n          157.0952241, \n          159.6062762\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\", \n        \"dec\": \"\", \n        \"properties\": {\n          \"dtype\": \"number\", \n          \"std\": 25.21220657928434, \n          \"min\": -5.382632499, \n          \"max\": 68.54226541, \n          \"num_unique_values\": 10000, \n          \"samples\": [\n            54.38634445, \n            62.59739268, \n            -0.017859983\n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\", \n          \"u\": \"\", \n          \"properties\": {\n            \"dtype\": \"number\", \n            \"std\": 0.8286560094113746, \n            \"min\": 12.98897, \n            \"max\": 19.5999, \n            \"num_unique_values\": 9730, \n            \"samples\": [\n              16.99432, \n              18.27211, \n              19.51903\n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\", \n            \"column\": \"

```

```

\"g\",\\n      \"properties\": {\\n      \"dtype\": \"number\",\\n
\"std\": 0.9454571752688135,\\n      \"min\": 12.79955,\\n
\"max\": 19.91897,\\n      \"num_unique_values\": 9817,\\n
\"samples\": [\\n      17.88955,\\n      18.89235,\\n
15.92992\\n      ],\\n      \"semantic_type\": \"\",\\n
\"description\": \"\"\\n      }\\n      },\\n      {\\n      \"column\":
\"r\",\\n      \"properties\": {\\n      \"dtype\": \"number\",\\n
\"std\": 1.0677643494029374,\\n      \"min\": 12.4316,\\n
\"max\": 24.80204,\\n      \"num_unique_values\": 9852,\\n
\"samples\": [\\n      17.62294,\\n      15.54049,\\n
18.64672\\n      ],\\n      \"semantic_type\": \"\",\\n
\"description\": \"\"\\n      }\\n      },\\n      {\\n      \"column\":
\"i\",\\n      \"properties\": {\\n      \"dtype\": \"number\",\\n
\"std\": 1.1418048192775534,\\n      \"min\": 11.94721,\\n
\"max\": 28.17963,\\n      \"num_unique_values\": 9890,\\n
\"samples\": [\\n      17.2252,\\n      18.65555,\\n
16.67307\\n      ],\\n      \"semantic_type\": \"\",\\n
\"description\": \"\"\\n      }\\n      },\\n      {\\n      \"column\":
\"z\",\\n      \"properties\": {\\n      \"dtype\": \"number\",\\n
\"std\": 1.2031880475759795,\\n      \"min\": 11.61041,\\n
\"max\": 22.83306,\\n      \"num_unique_values\": 9896,\\n
\"samples\": [\\n      17.83397,\\n      16.71616,\\n
17.0249\\n      ],\\n      \"semantic_type\": \"\",\\n
\"description\": \"\"\\n      }\\n      },\\n      {\\n      \"column\":
\"run\",\\n      \"properties\": {\\n      \"dtype\": \"number\",\\n
\"std\": 273,\\n      \"min\": 308,\\n      \"max\": 1412,\\n
\"num_unique_values\": 23,\\n      \"samples\": [\\n      1345,\\n
1332,\\n      752\\n      ],\\n      \"semantic_type\": \"\",\\n
\"description\": \"\"\\n      }\\n      },\\n      {\\n      \"column\":
\"rerun\",\\n      \"properties\": {\\n      \"dtype\": \"number\",\\n
\"std\": 0,\\n      \"min\": 301,\\n      \"max\": 301,\\n
\"num_unique_values\": 1,\\n      \"samples\": [\\n      301\\n
],\\n      \"semantic_type\": \"\",\\n      \"description\": \"\"\\n
}\\n      },\\n      {\\n      \"column\": \"camcol\",\\n      \"properties\":
{\\n      \"dtype\": \"number\",\\n      \"std\": 1,\\n
\"min\": 1,\\n      \"max\": 6,\\n      \"num_unique_values\": 6,\\n
\"samples\": [\\n      4\\n      ],\\n      \"semantic_type\":
\"\",\\n      \"description\": \"\"\\n      }\\n      },\\n      {\\n
\"column\": \"field\",\\n      \"properties\": {\\n      \"dtype\":
\"number\",\\n      \"std\": 162,\\n      \"min\": 11,\\n
\"max\": 768,\\n      \"num_unique_values\": 703,\\n
\"samples\": [\\n      621\\n      ],\\n      \"semantic_type\":
\"\",\\n      \"description\": \"\"\\n      }\\n      },\\n      {\\n
\"column\": \"class\",\\n      \"properties\": {\\n      \"dtype\":
\"number\",\\n      \"std\": 0,\\n      \"min\": 0,\\n
\"max\": 2,\\n      \"num_unique_values\": 3,\\n      \"samples\":
[\\n      2\\n      ],\\n      \"semantic_type\": \"\",\\n
\"description\": \"\"\\n      }\\n      },\\n      {\\n      \"column\":
\"redshift\",\\n      \"properties\": {\\n      \"dtype\":

```

```

\"number\",\\n          \"std\": 0.38877403581146536,\\n          \"min\": -
0.004136078,\\n          \"max\": 5.353854,\\n
\"num_unique_values\": 9637,\\n          \"samples\": [\\n
0.000189883\\n          ],\\n          \"semantic_type\": \"\",\\n
\"description\": \"\"\\n          }\\n          },\\n          {\\n          \"column\":
\"plate\",\\n          \"properties\": {\\n          \"dtype\": \"number\",\\n
\"std\": 1788,\\n          \"min\": 266,\\n          \"max\": 8410,\\n
\"num_unique_values\": 487,\\n          \"samples\": [\\n          3848\\n
],\\n          \"semantic_type\": \"\",\\n          \"description\": \"\"\\n
}\\n          },\\n          {\\n          \"column\": \"mjd\",\\n          \"properties\": {\\n
          \"dtype\": \"number\",\\n          \"std\": 1511,\\n
\"min\": 51578,\\n          \"max\": 57481,\\n
\"num_unique_values\": 355,\\n          \"samples\": [\\n          54862\\n
],\\n          \"semantic_type\": \"\",\\n          \"description\": \"\"\\n
}\\n          },\\n          {\\n          \"column\": \"fiberid\",\\n
\"properties\": {\\n          \"dtype\": \"number\",\\n          \"std\":
206,\\n          \"min\": 1,\\n          \"max\": 1000,\\n
\"num_unique_values\": 892,\\n          \"samples\": [\\n          658\\n
],\\n          \"semantic_type\": \"\",\\n          \"description\": \"\"\\n
}\\n          }\\n          ]\\n          }\", \"type\": \"dataframe\", \"variable_name\": \"data\"}

```

After lable encoding, Galaxies have been replaced by number 0, Quasars by number 1 and Stars by number 2

```

#Checking the information about the data
data.info()

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 16 columns):
#   Column      Non-Null Count  Dtype
---  -
0    ra           10000 non-null   float64
1    dec          10000 non-null   float64
2    u            10000 non-null   float64
3    g            10000 non-null   float64
4    r            10000 non-null   float64
5    i            10000 non-null   float64
6    z            10000 non-null   float64
7    run          10000 non-null   int64
8    rerun        10000 non-null   int64
9    camcol       10000 non-null   int64
10   field        10000 non-null   int64
11   class        10000 non-null   int64
12   redshift     10000 non-null   float64
13   plate        10000 non-null   int64
14   mjd          10000 non-null   int64
15   fiberid      10000 non-null   int64

```

```
dtypes: float64(8), int64(8)
memory usage: 1.2 MB
```

We can understand that the dataset is complete with no null values. And since the dataset has only numerical values now, there's no need of further encoding and type conversions.

Perform train and test split

```
X = data.drop('class', axis=1)
y = data['class']
```

Data Scaling

Scaling means converting floating-point feature values from their natural range (for example, 100 to 900) into a standard range—usually 0 and 1 (or sometimes -1 to +1). `StandardScaler` follows Standard Normal Distribution (SND). Therefore, it makes mean = 0 and scales the data to unit variance. You can read more about other types of scalers [here](#) and scaling in general [here](#).

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler(copy=True, with_mean=True, with_std=True)
X = scaler.fit_transform(X)
```

#Show data

```
X[:20]
```

```
array([[ 0.16745842, -0.58492272,  1.03148637, -0.34855938, -
 0.83728027,
        -0.94605772, -0.99534154, -0.83806089,  0.        ,
 0.21085172,
        -0.21763043, -0.36973112,  1.03148936,  1.30931064,
 0.66863177],
       [ 0.16886159, -0.58311429,  0.05243046, -0.16653251, -
 0.15415531,
        -0.08264457, -0.02604308, -0.83806089,  0.        ,
 0.21085172,
        -0.21763043, -0.36984929, -0.63621258, -0.87919741,
 0.91101156],
       [ 0.17057433, -0.58347525,  0.92156796,  0.86709322,
 0.59315368,
        0.44120145,  0.31452753, -0.83806089,  0.        ,
 0.21085172,
        -0.21147922, -0.05302706, -0.65633905, -0.60919097,
 0.77527888],
       [ 0.17455754, -0.58650069, -1.03063038, -0.81362749, -
 0.63669227,
        -0.52660429, -0.43092107, -0.83806089,  0.        ,
 0.21085172,
```

-0.20532801, -0.36999261, 1.03148936, 1.30931064,
0.76073609],
[0.17482457, -0.58441247, -1.29023238, -1.17251944, -
0.37676237,
-0.02510121, 0.15827647, -0.83806089, 0. ,
0.21085172,
-0.20532801, -0.36818949, 1.03148936, 1.30931064,
0.77043128],
[0.17406874, -0.58159078, 0.97991837, 1.15913585, 1.2396114
,
1.25349122, 1.34542376, -0.83806089, 0. ,
0.21085172,
-0.20532801, -0.36889881, -0.63565351, -0.8454466 ,
1.16793412],
[0.17442882, -0.58771882, 0.9218576 , 0.54793007,
0.24763882,
0.07037522, -0.0442872 , -0.83806089, 0. ,
0.21085172,
-0.20532801, -0.11185311, -0.65633905, -0.60919097,
0.99826828],
[0.17517603, -0.58104423, 0.43515363, 0.50034225,
0.50505868,
0.54578672, 0.53849374, -0.83806089, 0. ,
0.21085172,
-0.20532801, -0.36889818, 1.03148936, 1.30931064,
0.78497407],
[0.17568894, -0.58462313, -0.8607082 , -0.4233206 , -
0.15505443,
-0.0401308 , 0.04415727, -0.83806089, 0. ,
0.21085172,
-0.1991768 , -0.36947888, -0.63621258, -0.87919741,
1.17278172],
[0.17671255, -0.58524271, 0.06412468, 0.36155588,
0.61126719,
0.61244768, 0.66793136, -0.83806089, 0. ,
0.21085172,
-0.1991768 , -0.26550833, -0.65577999, -0.62441192,
0.22750057],
[0.17683178, -0.58308602, 0.81867332, 0.45518723,
0.32002733,
0.29758829, 0.31289013, -0.83806089, 0. ,
0.21085172,
-0.1991768 , -0.36979707, 1.03148936, 1.30931064,
0.74134571],
[0.17905309, -0.58403354, 0.25791803, 0.15434418,
0.09531295,
0.1143867 , 0.15158558, -0.83806089, 0. ,
0.21085172,
-0.19302559, -0.3681061 , 1.03148936, 1.30931064,

```
0.94009713],
  [ 0.179404 , -0.58088408, 1.13821874, 0.86650088,
0.76483872,
  0.78139043, 0.80556427, -0.83806089, 0. ,
0.21085172,
  -0.19302559, -0.36956736, 1.03148936, 1.30931064,
0.92555434],
  [ 0.18062649, -0.58547992, 1.15921765, 0.37678737,
0.13715012,
  0.08511578, 0.06767929, -0.83806089, 0. ,
0.21085172,
  -0.19302559, -0.36968672, 1.03148936, 1.30931064,
0.93524953],
  [ 0.18123476, -0.58453441, 0.76913279, 0.18696495, -
0.19217122,
  -0.38043364, -0.54561378, -0.83806089, 0. ,
0.21085172,
  -0.19302559, -0.18427654, -0.65577999, -0.62441192,
0.17417702],
  [ 0.18460582, -0.58026059, 0.14357054, 1.30915544,
1.45732014,
  1.51360998, 1.2914644 , -0.83806089, 0. ,
0.21085172,
  -0.18687438, 0.3297998 , -0.65633905, -0.60919097,
1.13400095],
  [ 0.18190916, -0.58661793, 0.53127794, 0.76494705,
1.01778171,
  1.1465937 , 1.2341887 , -0.83806089, 0. ,
0.21085172,
  -0.18687438, -0.36916795, -0.63565351, -0.8454466 , -
1.54187183],
  [ 0.18240867, -0.58061649, 0.72660397, 2.04202274,
2.15158106,
  2.21584041, 2.33699748, -0.83806089, 0. ,
0.21085172,
  -0.18687438, 2.66073327, -0.65633905, -0.60919097,
1.11461057],
  [ 0.18540578, -0.58577178, 0.51243926, 0.14716211,
0.03033282,
  0.02413909, 0.05003362, -0.83806089, 0. ,
0.21085172,
  -0.18072317, -0.36909046, -0.63621258, -0.87919741,
1.33275238],
  [ 0.18523939, -0.58156585, -0.96878016, -0.531168 , -
0.28296358,
  -0.33967158, -0.27939936, -0.83806089, 0. ,
0.21085172,
  -0.18072317, -0.18251527, -0.65633905, -0.60919097,
1.35214276]]))
```

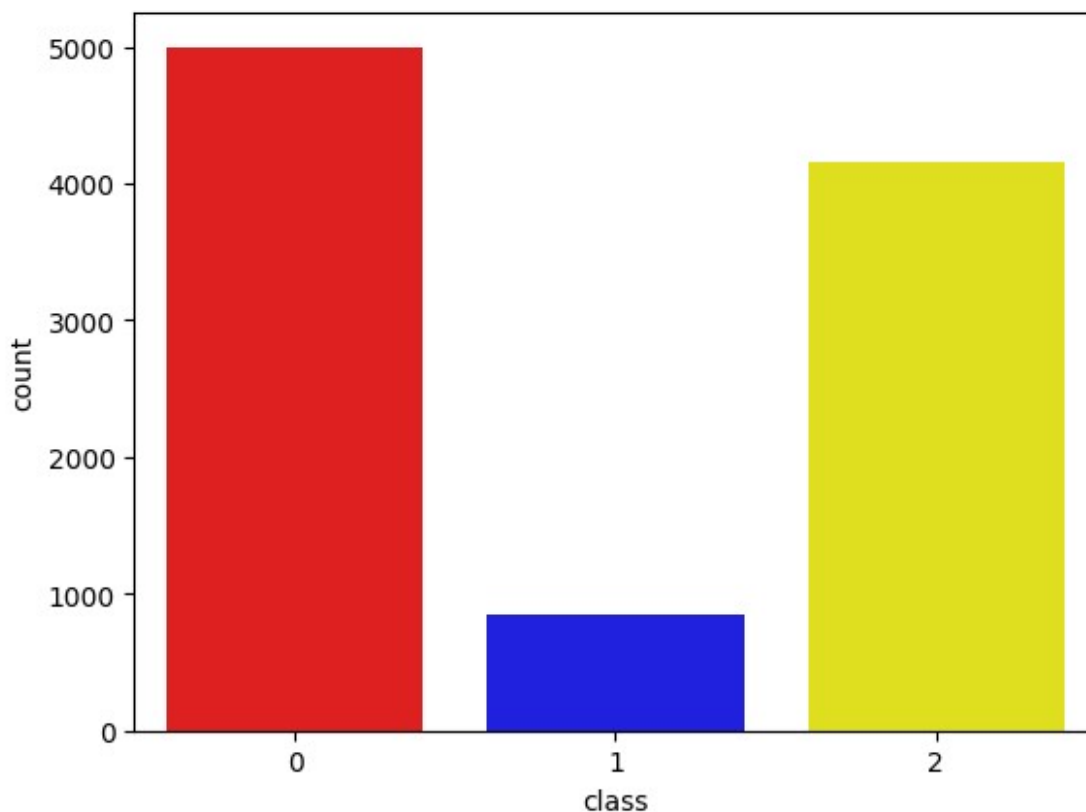
```
#Performing the 30% test and 70% train split here
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=128)
```

Density Distribution Plots

```
#Checking the number of labels for each class of the dataset where 0 =
Galaxy, 1 = Quasar, 2 = Star.
```

```
color_map = {'0': 'red', '1': 'blue', '2': 'yellow'}
sns.countplot(x=data['class'], palette=color_map)
```

```
<Axes: xlabel='class', ylabel='count'>
```



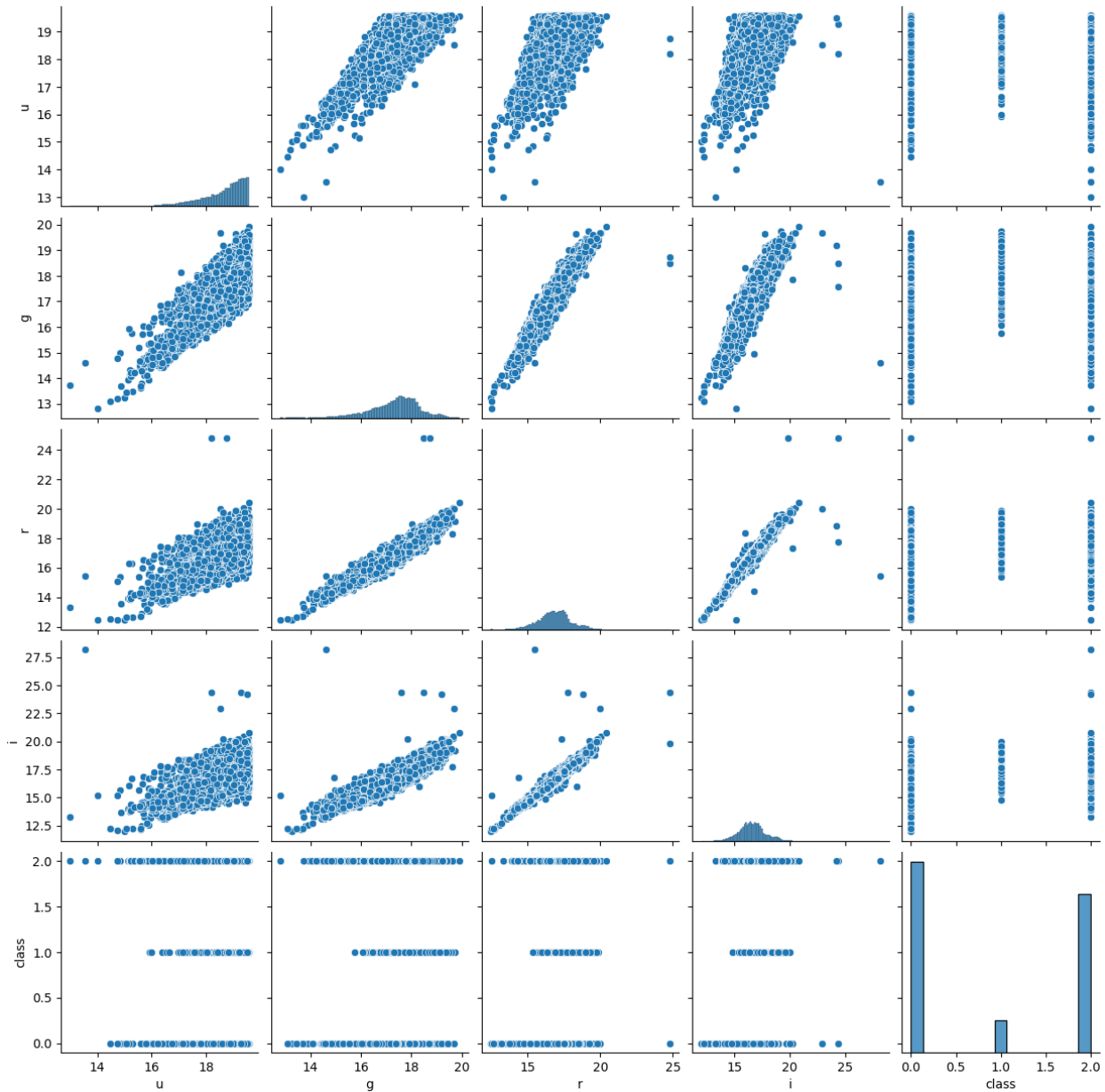
Some information about the filters used while gathering the data:

"U" stands for ultraviolet. "G" stands for green. "R" stands for red. "I" stands for infrared.

```
#Using pairplots to establish and understand interdependency of train
features
```

```
sns.pairplot(data[['u','g','r','i','class']])
```

```
<seaborn.axisgrid.PairGrid at 0x7d6a86e2ab40>
```



Diagonal Plots (Histograms/KDEs): These show the distribution of each individual feature (u, g, r, i). You can observe if the data is normally distributed, skewed, or multimodal. The different colors (representing 'Galaxy', 'Quasar', 'Star') will show how the distributions of each class overlap or separate for a single feature. For example, if 'Stars' tend to have higher 'u' values and 'Galaxies' lower, their colored distributions on the 'u' axis will be distinct.

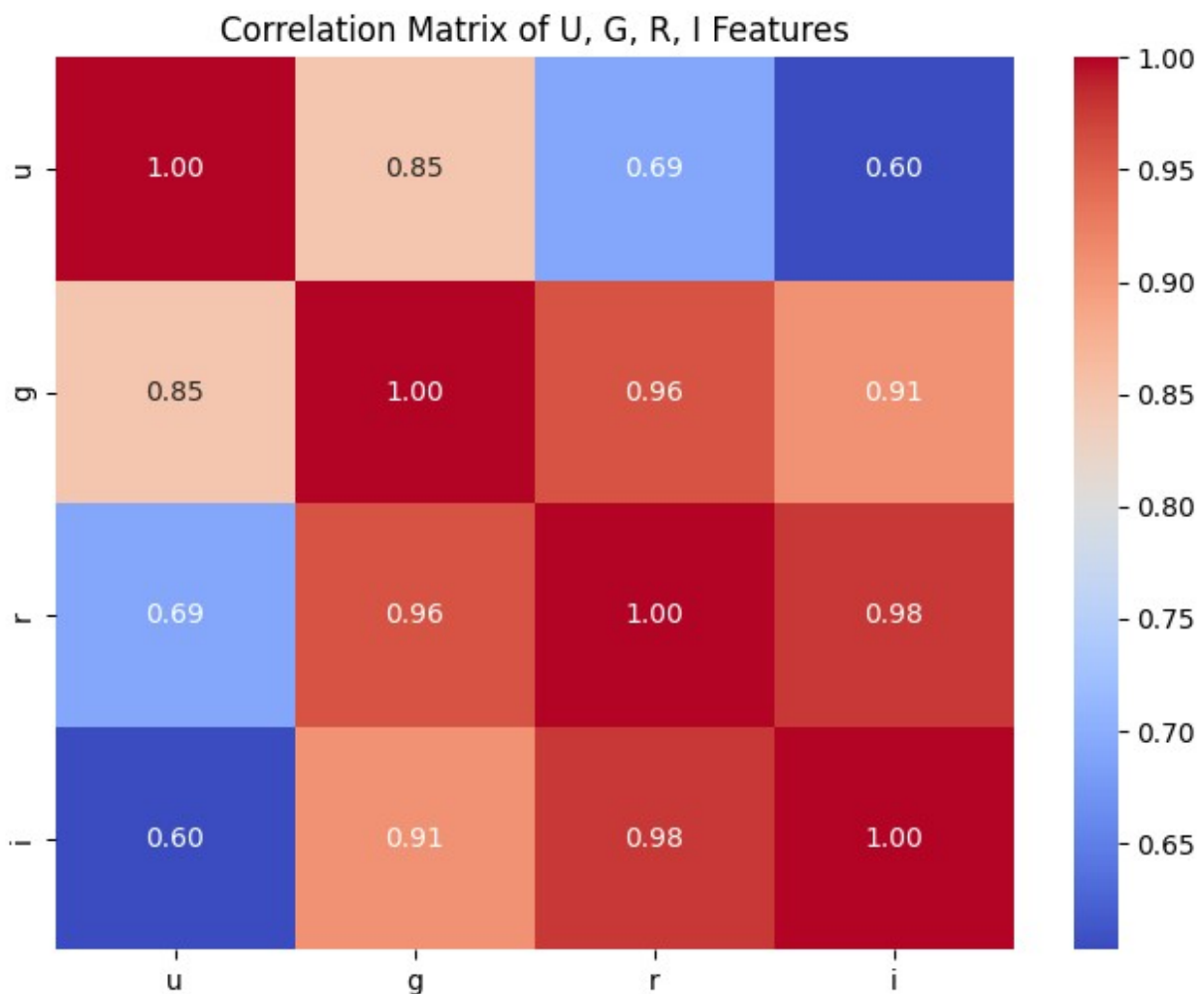
Off-Diagonal Plots (Scatter Plots): These show the relationship between each pair of features. Each point in a scatter plot represents an individual observation (celestial object). The color of the points indicates their class. You should look for:

Clusters: Do points of the same color (class) group together in certain regions of the scatter plots? This suggests that these two features, in combination, are good at distinguishing between classes. **Separation:** Are the different colored points well-separated or are they heavily

intermixed? Good separation indicates that the features are strong predictors. Correlations: Observe if there's a positive, negative, or no correlation between pairs of features. For instance, if 'u' values generally increase as 'g' values increase within a certain class, that indicates a positive correlation. In the context of astronomical objects, the features u, g, r, and i represent different photometric magnitudes (brightness measured through specific filters: ultraviolet, green, red, infrared). Analyzing their interdependencies and how they relate to the class (Star, Galaxy, Quasar) is crucial for understanding the distinct characteristics of these objects and how well these features can be used for classification.

```
correlation_matrix = data[['u', 'g', 'r', 'i']].corr()

plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm',
            fmt=".2f")
plt.title('Correlation Matrix of U, G, R, I Features')
plt.show()
```



The heatmap shows the correlation matrix for the 'u', 'g', 'r', and 'i' photometric features. The color intensity and the numeric values (ranging from -1 to 1) indicate the strength and direction of the linear relationship between each pair of features.

Positive Correlation (closer to 1, warmer colors): If two features have a high positive correlation, it means they tend to increase or decrease together. For example, a high correlation between 'g' and 'r' (0.96) suggests that objects that are brighter in the g-band also tend to be brighter in the r-band. **Negative Correlation** (closer to -1, cooler colors): A high negative correlation means that as one feature increases, the other tends to decrease.

No Correlation (closer to 0): A correlation near zero indicates little to no linear relationship between the features. From the heatmap, we can observe strong positive correlations among all four photometric bands (u, g, r, i), particularly between g, r, and i (all above 0.90). This is expected, as these magnitudes often measure the brightness of celestial objects in adjacent parts of the electromagnetic spectrum, and brighter objects tend to be brighter across multiple bands. Understanding these correlations can be useful for feature selection, as highly correlated features might contain redundant information.

Machine Learning models (Classification models)

Dbscan Clustering

```
from sklearn.cluster import DBSCAN

# Instantiate DBSCAN with example parameters. These may need tuning.
# eps: The maximum distance between two samples for one to be
# considered as in the neighborhood of the other.
# min_samples: The number of samples (or total weight) in a
# neighborhood for a point to be considered as a core point.
dbscan = DBSCAN(eps=1.0, min_samples=10)

# Fit the model and predict clusters
clusters = dbscan.fit_predict(X)

data['dbscan_cluster'] = clusters

# Number of clusters in labels, ignoring noise if present.
n_clusters_ = len(set(clusters)) - (1 if -1 in clusters else 0)
n_noise_ = list(clusters).count(-1)

print(f"Estimated number of clusters: {n_clusters_}")
print(f"Estimated number of noise points: {n_noise_}")

Estimated number of clusters: 34
Estimated number of noise points: 2685

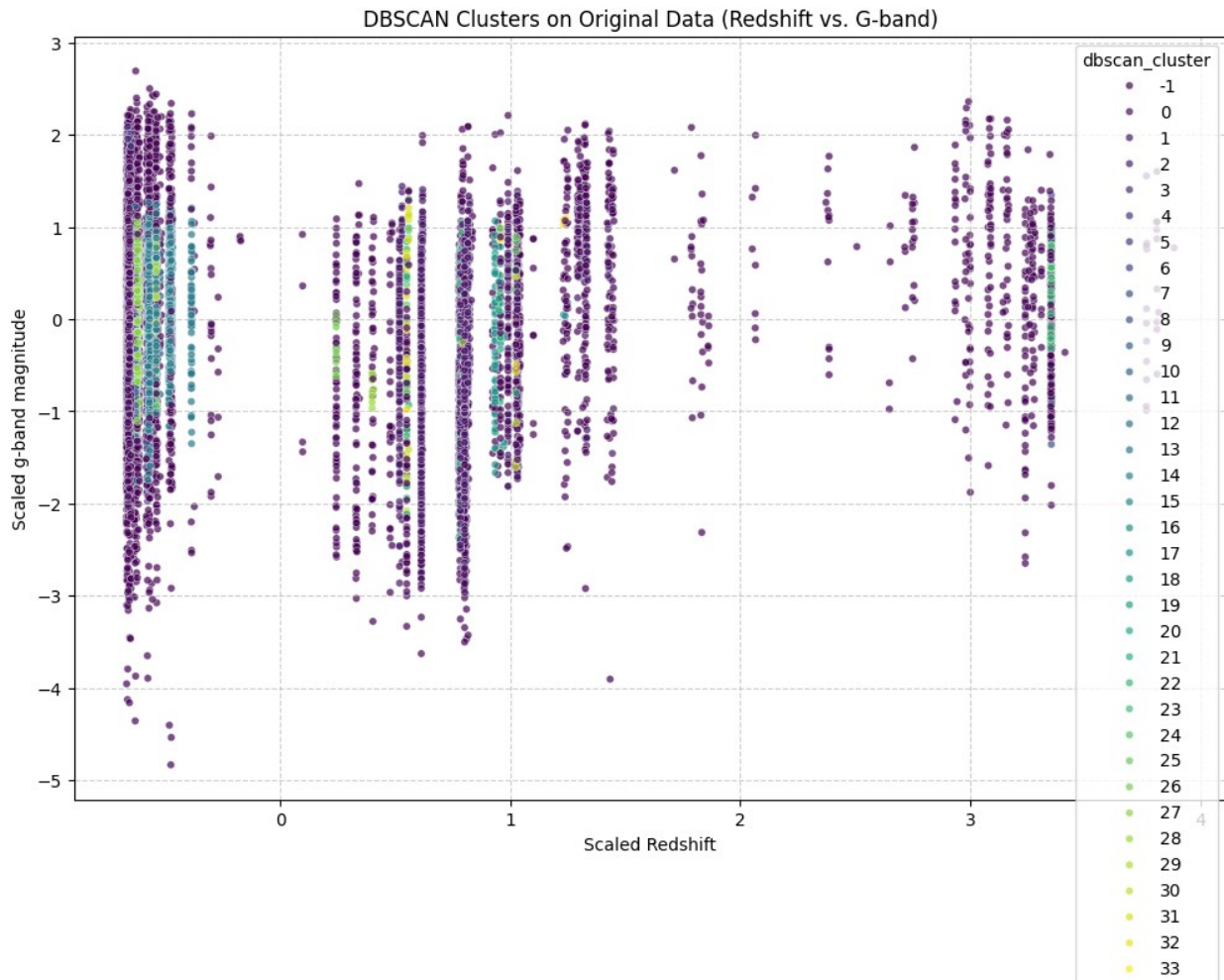
# Map numerical labels back to original class names for the legend
from `original_class_labels`
# We know from previous head() output that 0=GALAXY, 1=QS0, 2=STAR
```

```

# However, for DBSCAN, cluster labels are arbitrary integers, and -1
# indicates noise.
# We'll use a standard palette to distinguish clusters and a specific
# color for noise (-1).

plt.figure(figsize=(12, 8))
sns.scatterplot(
    x=X[:, data.columns.get_loc('redshift')],
    y=X[:, data.columns.get_loc('g')],
    hue=data['dbscan_cluster'],
    palette='viridis', # 'viridis' or 'Paired' are good choices for
many clusters
    legend='full',
    s=20, # Adjust marker size
    alpha=0.7 # Adjust transparency
)
plt.title('DBSCAN Clusters on Original Data (Redshift vs. G-band)')
plt.xlabel('Scaled Redshift')
plt.ylabel('Scaled g-band magnitude')
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()

```



Comparison: DBSCAN Clusters vs. Actual Classes (Galaxy, Quasar, Star)

```
# Assuming 'le' from previous LabelEncoder step is still available
original_class_labels =
le.inverse_transform(sorted(data['class'].unique()))

plt.figure(figsize=(12, 8))
sns.scatterplot(x=X[:, data.columns.get_loc('redshift')], y=X[:,
data.columns.get_loc('g')], hue=data['class'], palette='viridis',
legend='full')
plt.title('Actual Classes (Galaxy, Quasar, Star) - Redshift vs G-
band')
plt.xlabel('Scaled Redshift')
plt.ylabel('Scaled g-band magnitude')
plt.grid(True, linestyle='--', alpha=0.6)

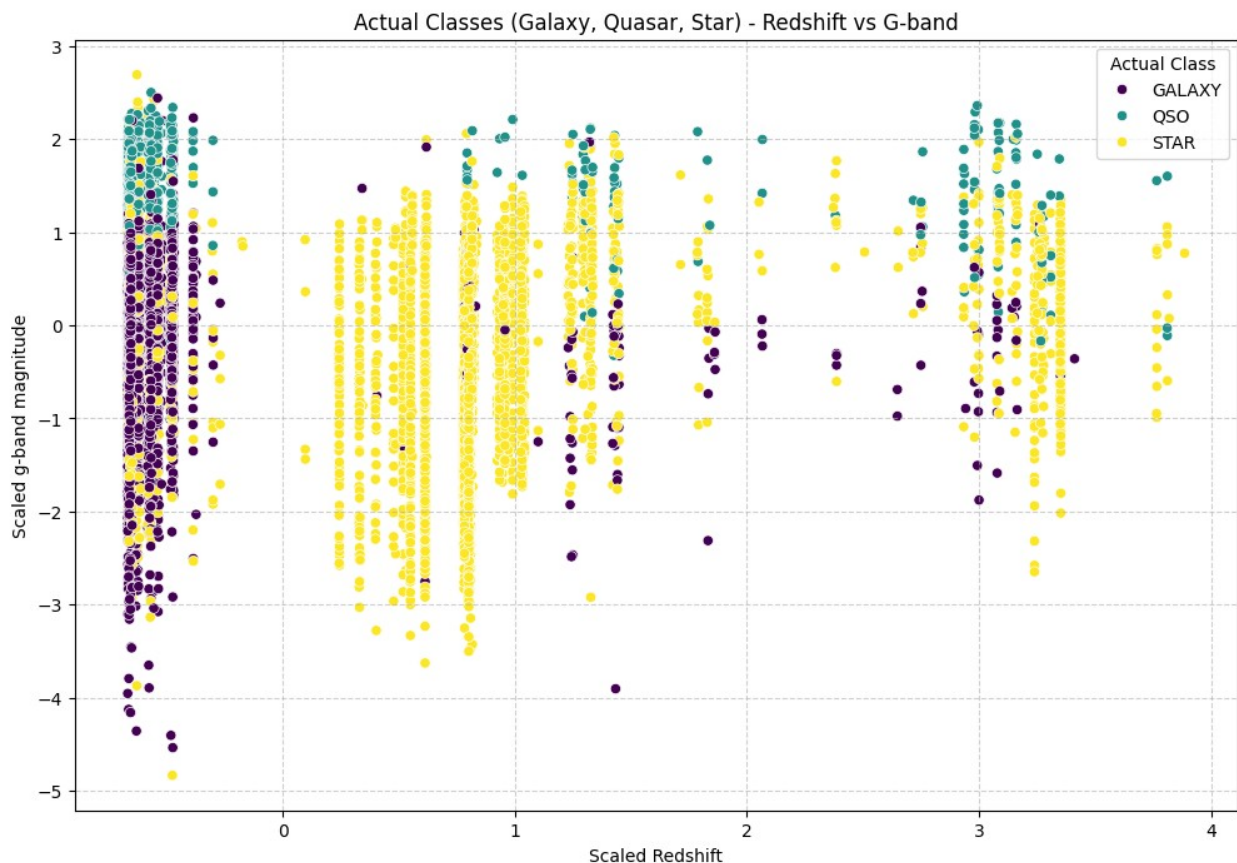
# Map numerical labels back to original class names for the legend
handles, labels = plt.gca().get_legend_handles_labels()
```

```

# Ensure the labels are in the correct order based on the numerical
mapping (0, 1, 2)
# We know from previous head() output that 0=GALAXY, 1=QSO, 2=STAR
new_labels = [original_class_labels[int(label)] for label in labels]
plt.legend(handles=handles, labels=new_labels, title='Actual Class')

plt.show()

```



```

from sklearn.cluster import KMeans

# Instantiate KMeans with 3 clusters (corresponding to GALAXY, QSO,
# STAR)
# random_state for reproducibility
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)

# Fit the model and predict clusters
clusters_kmeans = kmeans.fit_predict(X)

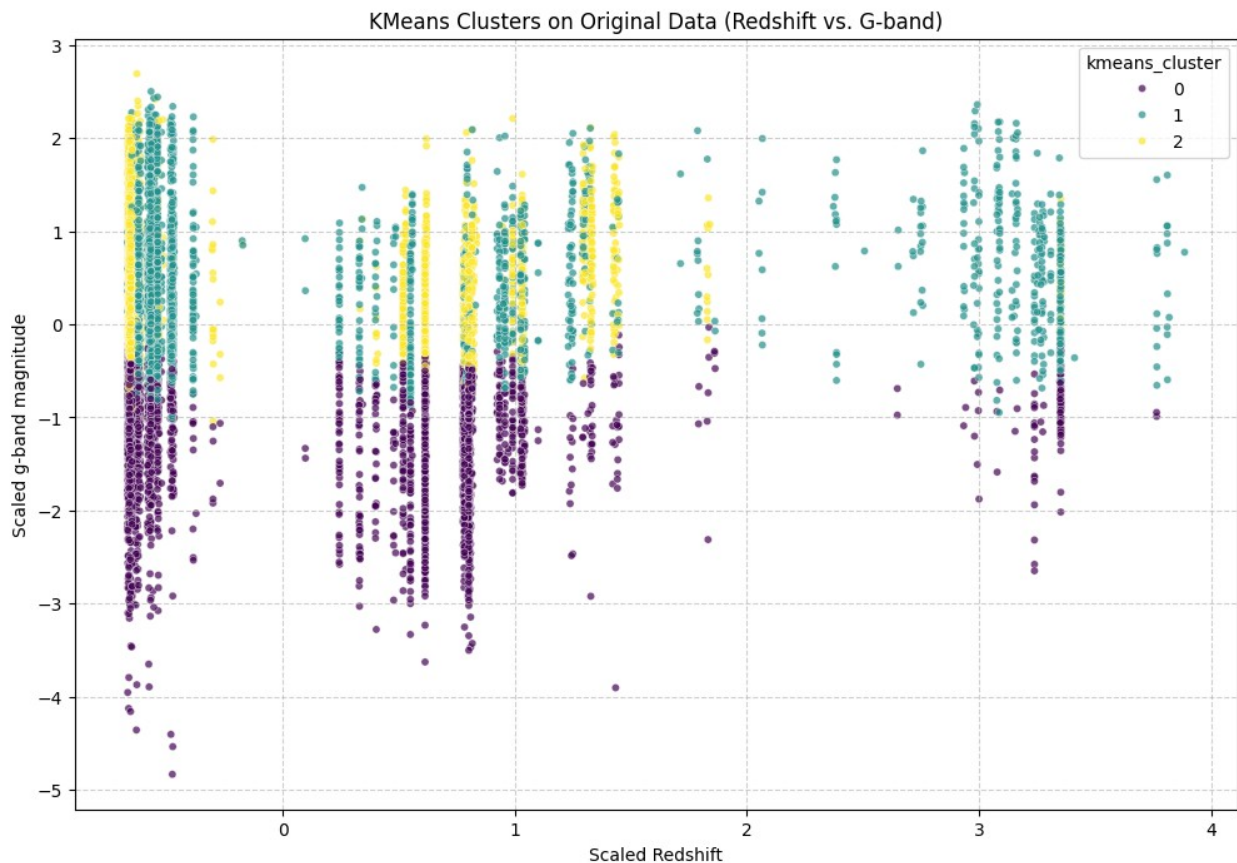
data['kmeans_cluster'] = clusters_kmeans

print(f"KMeans identified {kmeans.n_clusters} clusters.")
# KMeans does not identify noise points in the same way DBSCAN does.

```

KMeans identified 3 clusters.

```
plt.figure(figsize=(12, 8))
sns.scatterplot(
    x=X[:, data.columns.get_loc('redshift')],
    y=X[:, data.columns.get_loc('g')],
    hue=data['kmeans_cluster'],
    palette='viridis', # A good choice for distinct clusters
    legend='full',
    s=20, # Adjust marker size
    alpha=0.7 # Adjust transparency
)
plt.title('KMeans Clusters on Original Data (Redshift vs. G-band)')
plt.xlabel('Scaled Redshift')
plt.ylabel('Scaled g-band magnitude')
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```



```
from sklearn.metrics import silhouette_score

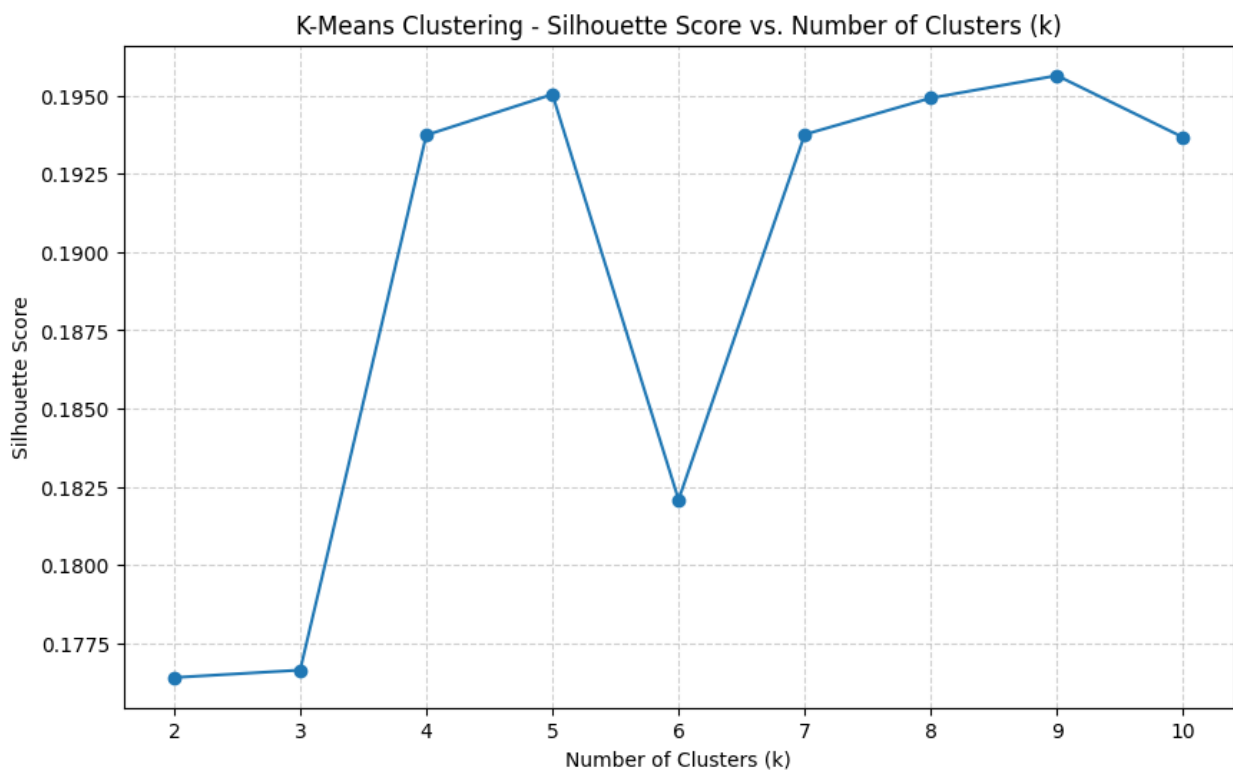
# Define a range of k values to test
k_values = range(2, 11) # K must be > 1 for silhouette score
silhouette_scores = []
```

```

for k in k_values:
    kmeans = KMeans(n_clusters=k, init='k-means++', random_state=42,
n_init=10)
    kmeans.fit(X)
    score = silhouette_score(X, kmeans.labels_)
    silhouette_scores.append(score)

# Plotting the silhouette scores
plt.figure(figsize=(10, 6))
plt.plot(k_values, silhouette_scores, marker='o', linestyle='-')
plt.title('K-Means Clustering - Silhouette Score vs. Number of
Clusters (k)')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Silhouette Score')
plt.xticks(k_values)
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()

```



```

from sklearn.cluster import KMeans
from sklearn.metrics import adjusted_rand_score, homogeneity_score,
completeness_score, v_measure_score

# Perform K-Means clustering (assuming k=3 from previous elbow method
analysis)

```

```

kmeans = KMeans(n_clusters=3, init='k-means++', random_state=42,
n_init=10)
clusters_kmeans = kmeans.fit_predict(X)
data['kmeans_cluster'] = clusters_kmeans

print("--- K-Means Clustering Performance ---")
print(f"Adjusted Rand Index (K-Means):
{adjusted_rand_score(data['class'], data['kmeans_cluster']):.4f}")
print(f"Homogeneity Score (K-Means): {homogeneity_score(data['class'],
data['kmeans_cluster']):.4f}")
print(f"Completeness Score (K-Means):
{completeness_score(data['class'], data['kmeans_cluster']):.4f}")
print(f"V-measure Score (K-Means): {v_measure_score(data['class'],
data['kmeans_cluster']):.4f}")

print("\n--- DBSCAN Clustering Performance ---")
# For DBSCAN, noise points (-1) are part of the clustering result and
should be included
print(f"Adjusted Rand Index (DBSCAN):
{adjusted_rand_score(data['class'], data['dbscan_cluster']):.4f}")
print(f"Homogeneity Score (DBSCAN): {homogeneity_score(data['class'],
data['dbscan_cluster']):.4f}")
print(f"Completeness Score (DBSCAN):
{completeness_score(data['class'], data['dbscan_cluster']):.4f}")
print(f"V-measure Score (DBSCAN): {v_measure_score(data['class'],
data['dbscan_cluster']):.4f}")

print("\n--- Summary of Clustering Performance ---")
if adjusted_rand_score(data['class'], data['kmeans_cluster']) >
adjusted_rand_score(data['class'], data['dbscan_cluster']):
    print("K-Means has a higher Adjusted Rand Index, indicating better
overall agreement with true labels.")
else:
    print("DBSCAN has a higher Adjusted Rand Index, indicating better
overall agreement with true labels.")

--- K-Means Clustering Performance ---
Adjusted Rand Index (K-Means): 0.0140
Homogeneity Score (K-Means): 0.0369
Completeness Score (K-Means): 0.0314
V-measure Score (K-Means): 0.0339

--- DBSCAN Clustering Performance ---
Adjusted Rand Index (DBSCAN): 0.1764
Homogeneity Score (DBSCAN): 0.4275
Completeness Score (DBSCAN): 0.1966
V-measure Score (DBSCAN): 0.2693

--- Summary of Clustering Performance ---

```

DBSCAN has a higher Adjusted Rand Index, indicating better overall agreement with true labels.

The Silhouette Score for DBSCAN, like for K-Means, provides insight into the compactness and separation of the clusters. A higher score indicates that objects are well-matched to their own cluster and poorly matched to neighboring clusters.

Since DBSCAN identifies noise points (labeled as -1), these are typically excluded from the silhouette score calculation as they don't belong to any cluster.

Decision tree classifier -

Decision trees use multiple algorithms to decide to split a node into two or more sub-nodes. The creation of sub-nodes increases the homogeneity of resultant sub-nodes. In other words, we can say that the purity of the node increases with respect to the target variable. The decision tree splits the nodes on all available variables and then selects the split which results in most homogeneous sub-nodes.

The algorithm selection is also based on the type of target variables. Let us look at some algorithms used in Decision Trees:

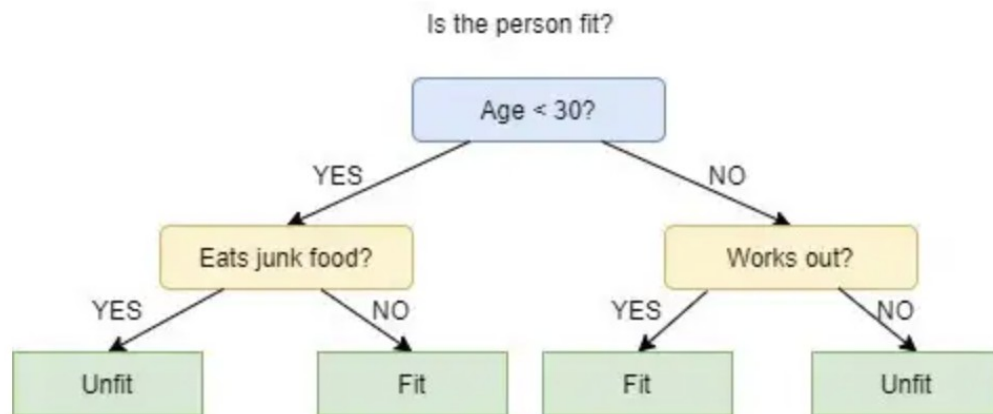
1. ID3 → (extension of D3)
2. C4.5 → (successor of ID3)
3. CART → (Classification And Regression Tree)
4. CHAID → (Chi-square automatic interaction detection Performs multi-level splits when computing classification trees)
5. MARS → (multivariate adaptive regression splines)

To understand these, let's have a look at the most primitive of these - ID3.

[Link for the example.](#)

You can read further about the methods [here](#).

Let's consider this example -



The initial node is called the root node (colored in blue), the final nodes are called the leaf nodes (colored in green) and the rest of the nodes are called intermediate or internal nodes. The root and intermediate nodes represent the decisions while the leaf nodes represent the outcomes.

ID3 stands for Iterative Dichotomiser 3 and is named such because the algorithm iteratively (repeatedly) dichotomizes(divides) features into two or more groups at each step. Invented by Ross Quinlan, ID3 uses a top-down greedy approach to build a decision tree. In simple words, the top-down approach means that we start building the tree from the top and the greedy approach means that at each iteration we select the best feature at the present moment to create a node. Most generally ID3 is only used for classification problems with nominal features only.

ID3 uses Information Gain or just Gain to find the best feature. Information Gain calculates the reduction in the entropy and measures how well a given feature separates or classifies the target classes. The feature with the highest Information Gain is selected as the best one.

In simple words, Entropy is the measure of disorder and the Entropy of a dataset is the measure of disorder in the target feature of the dataset. In the case of binary classification (where the target column has only two types of classes) entropy is 0 if all values in the target column are homogenous(similar) and will be 1 if the target column has equal number values for both the classes.

ID3 Steps -

1. Calculate the Information Gain of each feature.
2. Considering that all rows don't belong to the same class, split the dataset S into subsets using the feature for which the Information Gain is maximum.
3. Make a decision tree node using the feature with the maximum Information gain.
4. If all rows belong to the same class, make the current node as a leaf node with the class as its label.
5. Repeat for the remaining features until we run out of all features, or the decision tree has all leaf nodes.

We denote our dataset as S , entropy is calculated as -

$$\text{Entropy}(S) = - \sum p_i * \log_2(p_i) ; i = 1 \text{ to } n$$

Where n is the total number of classes in the target column; p_i is the probability of class 'i' or the ratio of "number of rows with class i in the target column" to the "total number of rows" in the dataset.

Information Gain for a feature column A is calculated as -

$$\text{IG}(S, A) = \text{Entropy}(S) - \sum ((|S_v| / |S|) * \text{Entropy}(S_v))$$

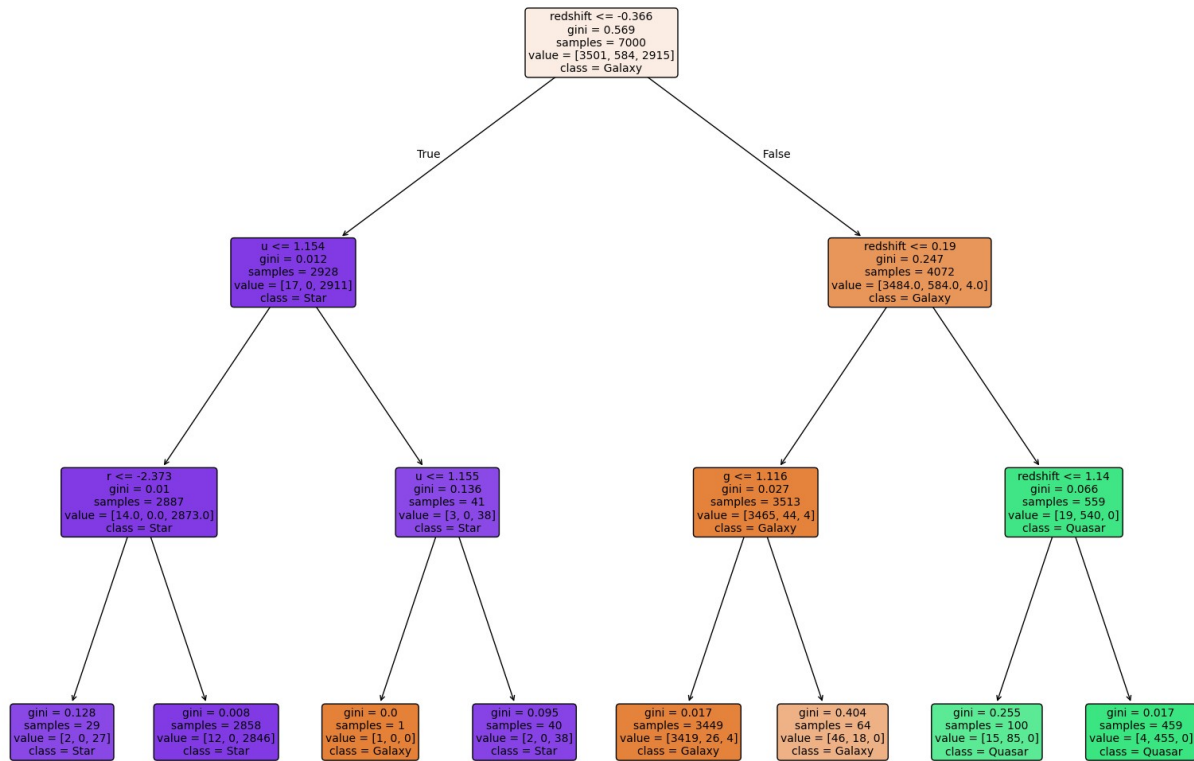
where S_v is the set of rows in S for which the feature column A has value v , $|S_v|$ is the number of rows in S_v and likewise $|S|$ is the number of rows in S .

```
# Decision Tree Classifier
dtc = DecisionTreeClassifier(max_leaf_nodes=15, max_depth=3)

from sklearn.tree import plot_tree

plt.figure(figsize=(20, 15))
plot_tree(dtc,
          feature_names=data.drop('class', axis=1).columns.tolist(),
          class_names=['Galaxy', 'Quasar', 'Star'],
          filled=True,
          rounded=True,
          fontsize=10)
plt.title('Decision Tree Classifier Visualization', fontsize=20)
plt.show()
```

Decision Tree Classifier Visualization



In the visualization above, each node in the Decision Tree represents a decision rule based on a specific feature (e.g., `redshift`, `g`).

- **Feature at Node:** Indicates which feature is being evaluated at that split point.
- **Threshold:** The value of the feature at which the split occurs (e.g., `redshift <= 0.093`).
- **gini:** The Gini impurity, which measures the impurity of the node. A lower Gini value means a purer node (more samples of a single class).
- **samples:** The number of training samples that reach this node.
- **value:** The distribution of classes among the samples at that node (e.g., `[4697, 439, 4801]` means 4697 samples of class 0, 439 of class 1, and 4801 of class 2).
- **class:** The predicted class for the majority of samples in that node.

By following the branches of the tree, you can see how different feature values lead to different classification outcomes, ultimately predicting whether an object is a Galaxy, Quasar, or Star.

```

feature_importances = dtc.feature_importances_
# Explicitly define feature_names to match the 15 features the model
was trained on
feature_names = ['ra', 'dec', 'u', 'g', 'r', 'i', 'z', 'run', 'rerun',

```

```

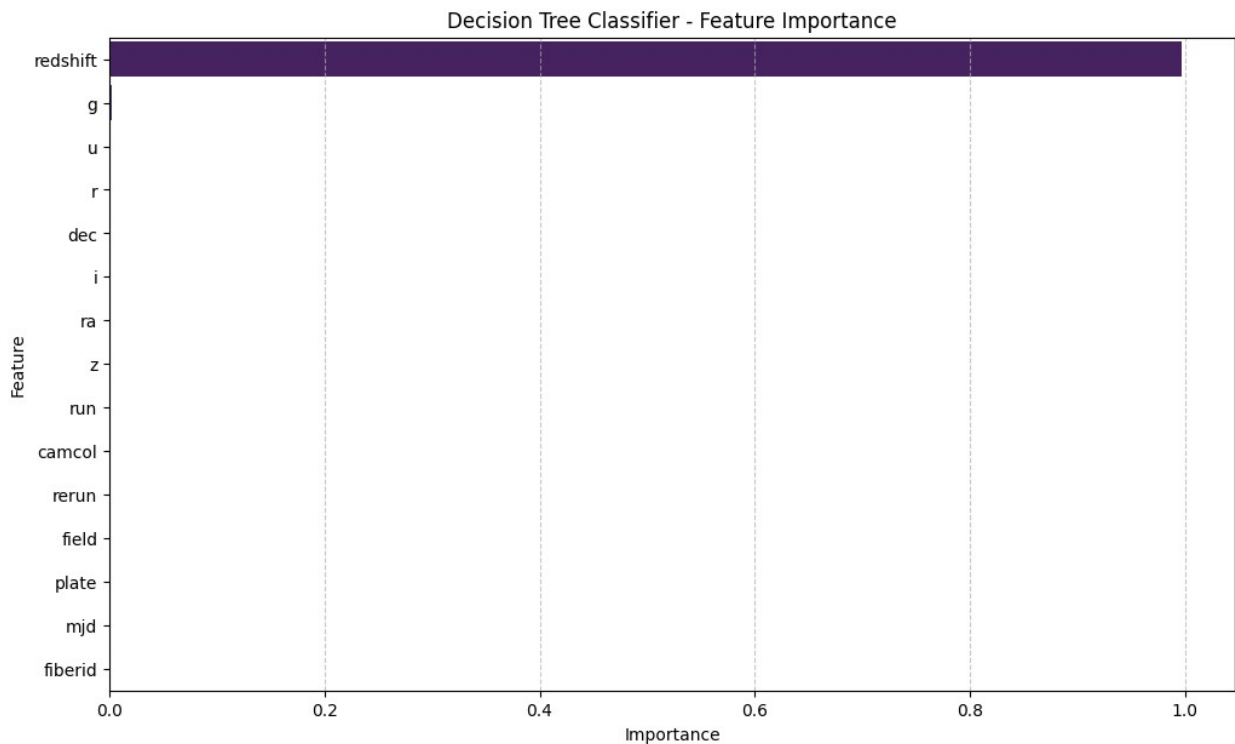
'camcol', 'field', 'redshift', 'plate', 'mjd', 'fiberid']

# Create a DataFrame for better visualization
importance_df = pd.DataFrame({
    'Feature': feature_names,
    'Importance': feature_importances
})

# Sort features by importance
importance_df = importance_df.sort_values(by='Importance',
ascending=False)

plt.figure(figsize=(12, 7))
sns.barplot(x='Importance', y='Feature', data=importance_df,
palette='viridis')
plt.title('Decision Tree Classifier - Feature Importance')
plt.xlabel('Importance')
plt.ylabel('Feature')
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.show()

```



```

from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

y_pred_dt = dtc.predict(X_test)

```

```

print(f"Decision Tree Classifier Accuracy: {accuracy_score(y_test,
y_pred_dt):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_dt))

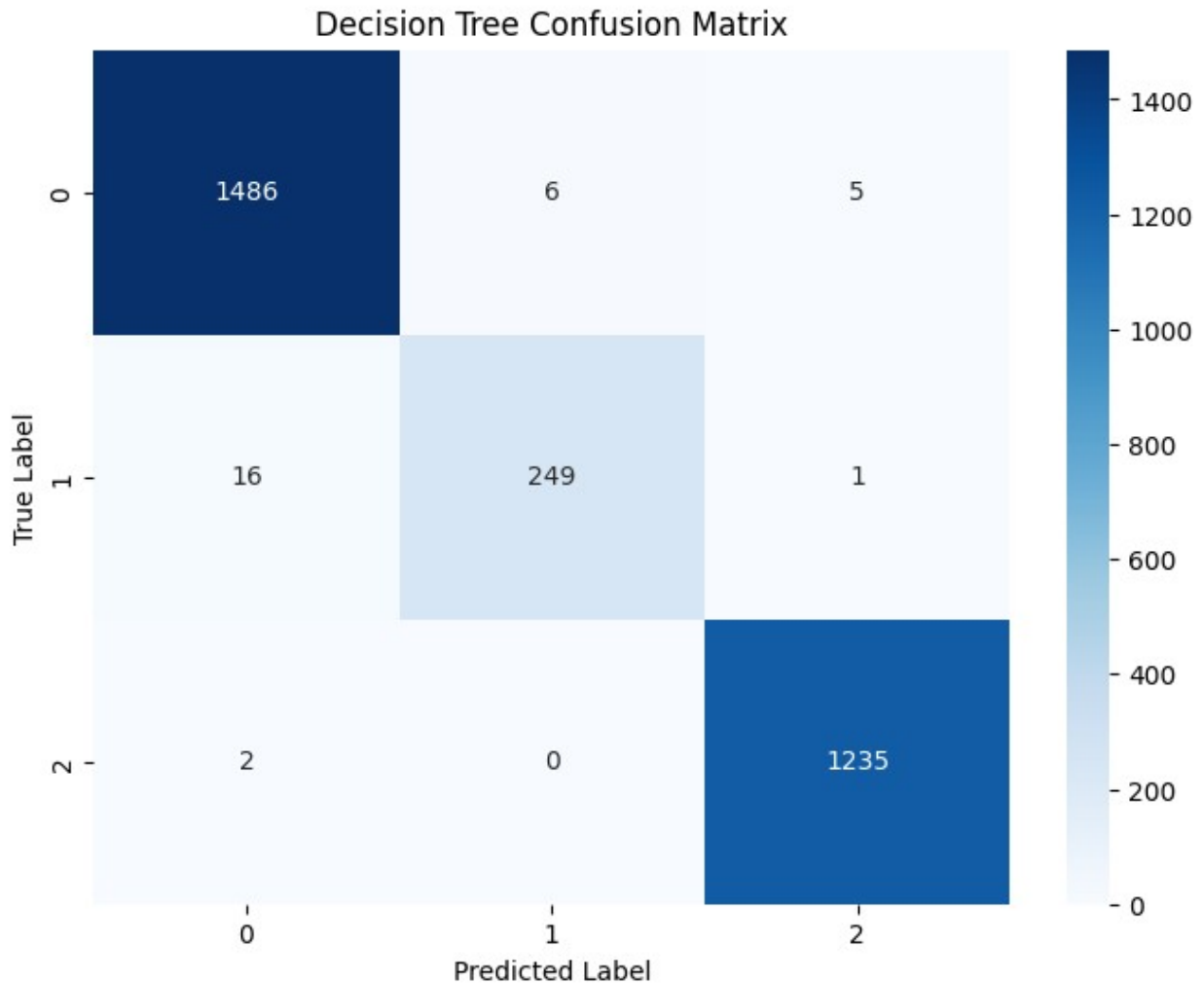
plt.figure(figsize=(8, 6))
sns.heatmap(confusion_matrix(y_test, y_pred_dt), annot=True, fmt='d',
cmap='Blues')
plt.title('Decision Tree Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()

```

Decision Tree Classifier Accuracy: 0.9900

Classification Report:

	precision	recall	f1-score	support
0	0.99	0.99	0.99	1497
1	0.98	0.94	0.96	266
2	1.00	1.00	1.00	1237
accuracy			0.99	3000
macro avg	0.99	0.98	0.98	3000
weighted avg	0.99	0.99	0.99	3000



```
from sklearn.model_selection import cross_val_score
cv_scores = cross_val_score(dtc, X, y, cv=5)
print(f"Decision Tree Classifier Cross-Validation Scores: {cv_scores}")
print(f"Mean CV Score: {cv_scores.mean():.4f}")
print(f"Standard Deviation of CV Scores: {cv_scores.std():.4f}")
```

Decision Tree Classifier Cross-Validation Scores: [0.99 0.99 0.9885 0.983 0.9865]
Mean CV Score: 0.9876
Standard Deviation of CV Scores: 0.0026

Interpretation of Decision Tree Classifier Performance

The Decision Tree Classifier demonstrated excellent performance in classifying stars, galaxies, and quasars. Here's a breakdown of its performance:

- **High Accuracy:** It achieved an accuracy of **0.9900** on the test set, indicating that 99% of the predictions were correct.
- **Strong Classification Metrics:** The classification report shows very high precision, recall, and f1-scores for all three classes (Galaxy, Quasar, and Star). Specifically:
 - **Galaxies (0):** 0.99 precision, 0.99 recall, 0.99 f1-score
 - **Quasars (1):** 0.98 precision, 0.94 recall, 0.96 f1-score
 - **Stars (2):** 1.00 precision, 1.00 recall, 1.00 f1-score This indicates that the model is highly effective at identifying each class, with particularly strong performance for 'Star' classification.
- **Consistent Cross-Validation Scores:** The 5-fold cross-validation resulted in a mean accuracy of **0.9876** with a very low standard deviation of **0.0026**. This suggests that the model's performance is robust and generalizes well across different subsets of the data, rather than being overly dependent on a specific train-test split.
- **Feature Importance:** The feature importance plot highlighted that certain features (likely related to magnitudes and redshift, given the nature of the data) were crucial for the Decision Tree to make its classifications, indicating their strong predictive power.

In summary, the Decision Tree Classifier is a highly effective and reliable model for this classification task, offering both high accuracy and consistency.

Logistic Regression

```
LRClassifier = LogisticRegression()
LRClassifier.fit(X_train,y_train)

LogisticRegression()

y_pred_lr = LRClassifier.predict(X_test)

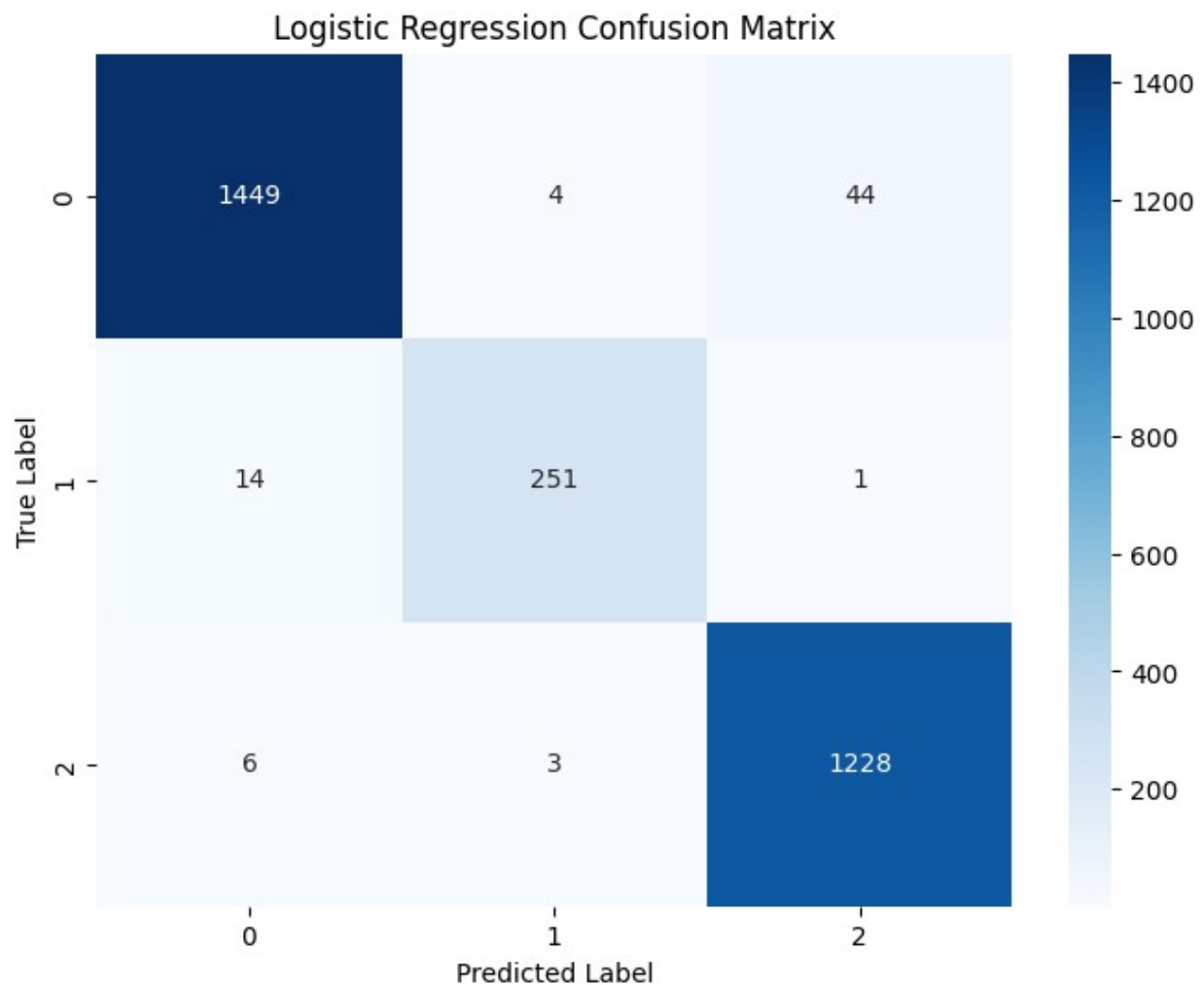
print(f"Logistic Regression Classifier Accuracy:
{accuracy_score(y_test, y_pred_lr):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_lr))
plt.figure(figsize=(8, 6))
sns.heatmap(confusion_matrix(y_test, y_pred_lr), annot=True, fmt='d',
cmap='Blues')
plt.title('Logistic Regression Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

Logistic Regression Classifier Accuracy: 0.9760

Classification Report:

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

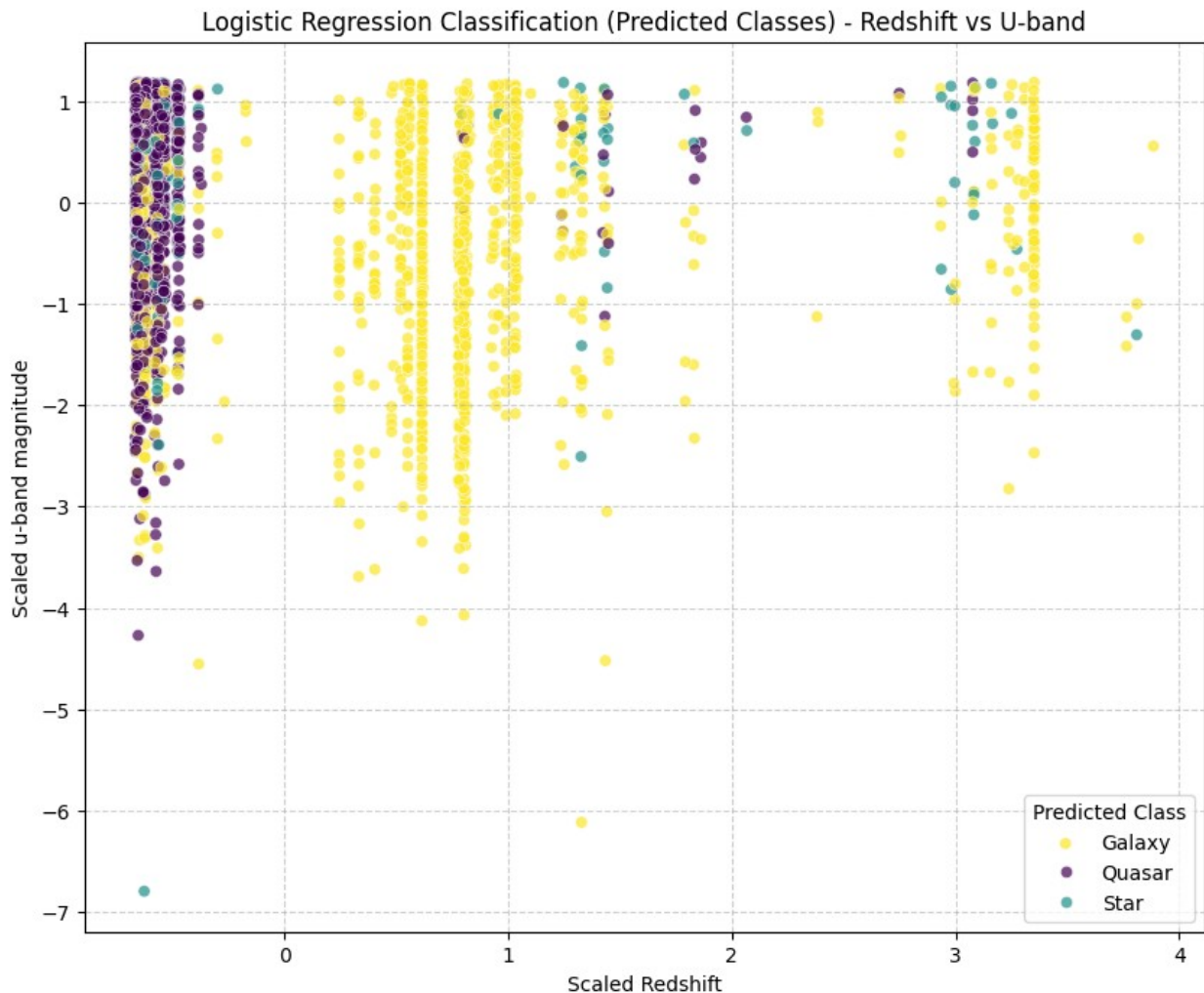
	0	0.99	0.97	0.98	1497
	1	0.97	0.94	0.96	266
	2	0.96	0.99	0.98	1237
accuracy				0.98	3000
macro avg		0.97	0.97	0.97	3000
weighted avg		0.98	0.98	0.98	3000



Visualizing Logistic Regression Classification

```
plt.figure(figsize=(10, 8))
sns.scatterplot(x=X_test[:, data.columns.get_loc('redshift')],
y=X_test[:, data.columns.get_loc('u')], hue=y_pred_lr,
palette='viridis', alpha=0.7)
plt.title('Logistic Regression Classification (Predicted Classes) - Redshift vs U-band')
```

```
plt.xlabel('Scaled Redshift')
plt.ylabel('Scaled u-band magnitude')
plt.legend(title='Predicted Class', labels=['Galaxy', 'Quasar',
'Star'])
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```



```
cv_scores_lr = cross_val_score(LRClassifier, X, y, cv=5)

print(f"Logistic Regression Classifier Cross-Validation Scores:
{cv_scores_lr}")
print(f"Mean LR CV Score: {cv_scores_lr.mean():.4f}")
print(f"Standard Deviation of LR CV Scores: {cv_scores_lr.std():.4f}")

Logistic Regression Classifier Cross-Validation Scores: [0.9775 0.981
0.976 0.971 0.977 ]
Mean LR CV Score: 0.9765
Standard Deviation of LR CV Scores: 0.0032
```

Interpretation of Logistic Regression Classifier Performance

The Logistic Regression Classifier also demonstrated strong performance, albeit slightly less than the Decision Tree, for classifying stars, galaxies, and quasars. Here's an overview:

- **Good Accuracy:** It achieved an accuracy of **0.9760** on the test set, meaning nearly 98% of its predictions were correct.
- **Solid Classification Metrics:** The classification report shows good precision, recall, and f1-scores across all three classes:
 - **Galaxies (0):** 0.99 precision, 0.97 recall, 0.98 f1-score
 - **Quasars (1):** 0.97 precision, 0.94 recall, 0.96 f1-score
 - **Stars (2):** 0.96 precision, 0.99 recall, 0.98 f1-score This indicates that the model is effective at distinguishing between the classes, with 'Star' classification being particularly strong in recall.
- **Consistent Cross-Validation Scores:** The 5-fold cross-validation resulted in a mean accuracy of **0.9765** with a standard deviation of **0.0032**. This consistency suggests that the model's performance is stable and generalizes well across different data subsets.

In summary, the Logistic Regression Classifier is a robust model for this classification task, offering high accuracy and reliability, though it was marginally outperformed by the Decision Tree in this specific evaluation.

KNN classifier -

The KNN algorithm assumes that similar things exist in close proximity. In other words, similar things are near to each other.

"Birds of a feather flock together."

Image showing how similar data points typically exist close to each other Notice in the image above that most of the time, similar data points are close to each other. The KNN algorithm hinges on this assumption being true enough for the algorithm to be useful. KNN captures the idea of similarity (sometimes called distance, proximity, or closeness) with some mathematics we might have learned in our childhood— calculating the distance between points on a graph.

We have explained KNN in much more detail in our previous project on [Hunting for Exoplanets](#).

Interpretation of K-Nearest Neighbors (KNN) Classifier Performance

Let's analyze the performance of the K-Nearest Neighbors (KNN) classifier:

- **Cross-Validation Scores:** The 5-fold cross-validation scores (**[0.896, 0.913, 0.8875, 0.883, 0.8835]**) show that KNN's accuracy ranges from about 88% to 91%. The mean CV score is **0.8926** with a standard deviation of **0.0112**. This

indicates a moderate level of consistency across different data subsets, but with a bit more variability compared to the Decision Tree and Logistic Regression.

- **Confusion Matrix:** The confusion matrix visually represents the model's predictions against the true labels for the test set. It helps us understand where the model is making errors. (The specific accuracy metrics from the classification report will give a more detailed breakdown for each class).
- **Comparison to Other Models:** While showing decent performance, the KNN classifier's mean cross-validation accuracy of 0.8926 is lower than that of both the Decision Tree (0.9876) and Logistic Regression (0.9765) classifiers. This suggests that for this particular dataset and current parameter settings, KNN is less effective at distinguishing between the classes compared to the other two models evaluated so far.

In summary, KNN provides reasonable classification but does not achieve the high accuracy and robustness observed with the Decision Tree and Logistic Regression for this dataset.

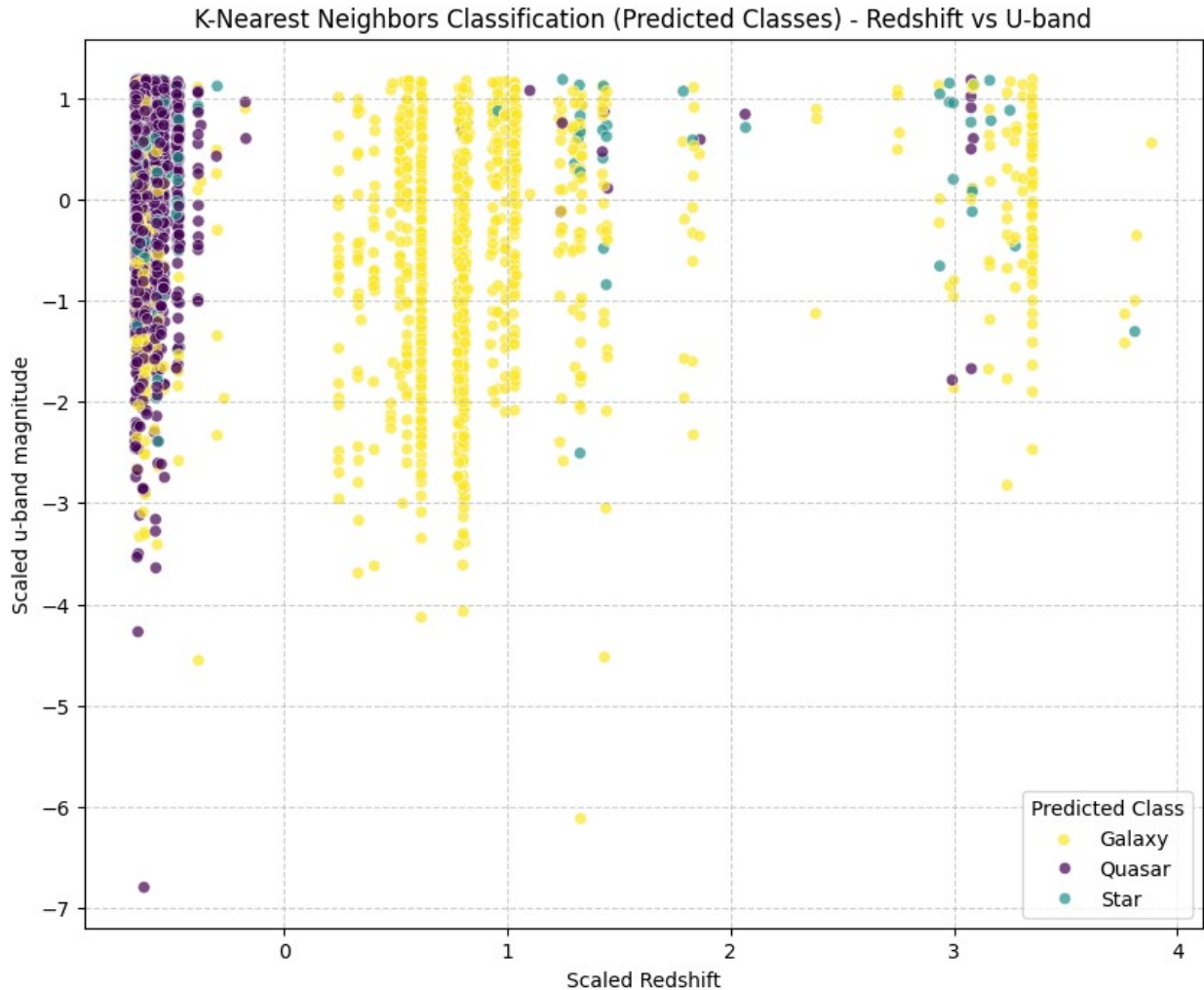
```
#Fitting the models to the dataset
dtc.fit(X_train, y_train)

LRClassifier.fit(X_train, y_train)

#Nearest Neighbor Classifier
NeNeClassifier = KNeighborsClassifier(n_neighbors=3)
NeNeClassifier.fit(X_train, y_train)

KNeighborsClassifier(n_neighbors=3)

plt.figure(figsize=(10, 8))
sns.scatterplot(x=X_test[:, data.columns.get_loc('redshift')],
y=X_test[:, data.columns.get_loc('u')], hue=y_predsNeNe,
palette='viridis', alpha=0.7)
plt.title('K-Nearest Neighbors Classification (Predicted Classes) -
Redshift vs U-band')
plt.xlabel('Scaled Redshift')
plt.ylabel('Scaled u-band magnitude')
plt.legend(title='Predicted Class', labels=['Galaxy', 'Quasar',
'Star'])
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```



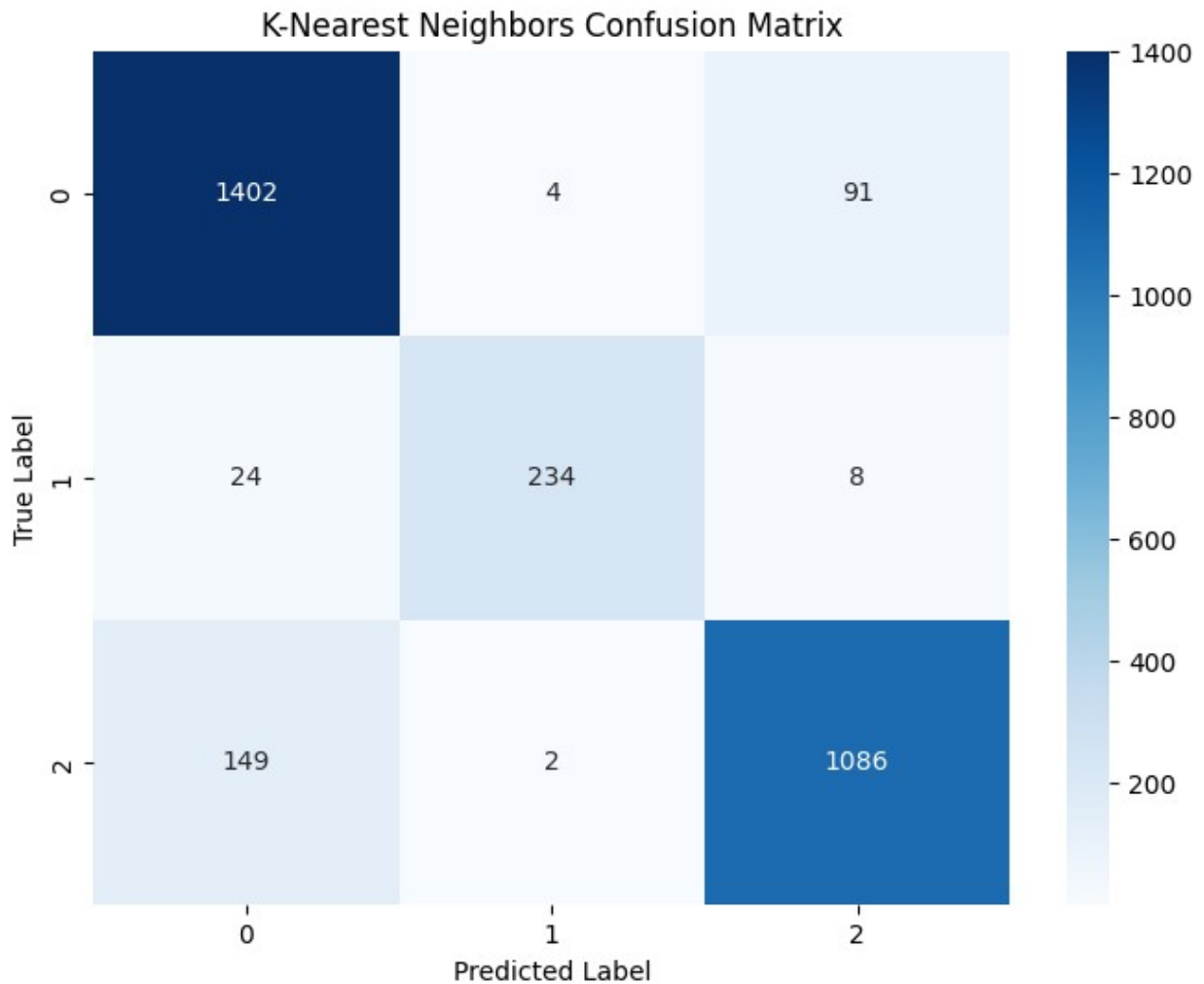
```
y_predsNeNe = NeNeClassifier.predict(X_test)
print("KNN Classifier Accuracy: {0:.4f}".format(accuracy_score(y_test,
y_predsNeNe)))
print("\nClassification Report:")
print(classification_report(y_test, y_predsNeNe))
```

KNN Classifier Accuracy: 0.9073

Classification Report:

	precision	recall	f1-score	support
0	0.89	0.94	0.91	1497
1	0.97	0.88	0.92	266
2	0.92	0.88	0.90	1237
accuracy			0.91	3000
macro avg	0.93	0.90	0.91	3000
weighted avg	0.91	0.91	0.91	3000

```
plt.figure(figsize=(8, 6))
sns.heatmap(confusion_matrix(y_test, y_predsNeNe), annot=True,
            fmt='d', cmap='Blues')
plt.title('K-Nearest Neighbors Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```



Random Forest

```
from sklearn.ensemble import RandomForestClassifier

# Instantiate and train the Random Forest Classifier
rf_classifier = RandomForestClassifier(n_estimators=100,
                                     random_state=42)
rf_classifier.fit(X_train, y_train)
```

```

# Make predictions on the test set
y_pred_rf = rf_classifier.predict(X_test)

# Evaluate Random Forest performance
print(f"Random Forest Classifier Accuracy: {accuracy_score(y_test,
y_pred_rf):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_rf))

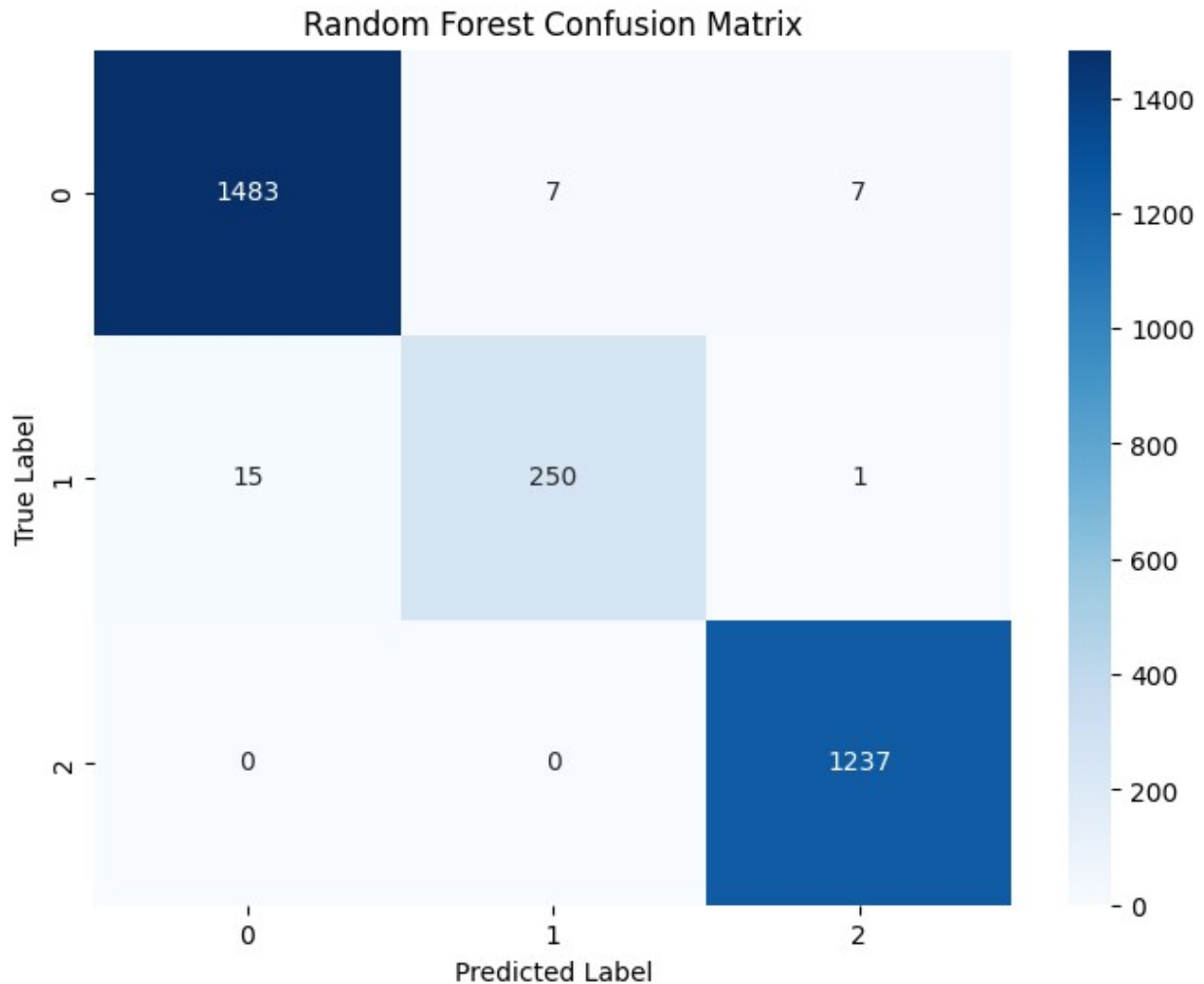
# Plot Confusion Matrix
plt.figure(figsize=(8, 6))
sns.heatmap(confusion_matrix(y_test, y_pred_rf), annot=True, fmt='d',
cmap='Blues')
plt.title('Random Forest Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()

```

Random Forest Classifier Accuracy: 0.9900

Classification Report:

	precision	recall	f1-score	support
0	0.99	0.99	0.99	1497
1	0.97	0.94	0.96	266
2	0.99	1.00	1.00	1237
accuracy			0.99	3000
macro avg	0.99	0.98	0.98	3000
weighted avg	0.99	0.99	0.99	3000



```
# Perform cross-validation for Random Forest
cv_scores_rf = cross_val_score(rf_classifier, X, y, cv=5)

print(f"Random Forest Classifier Cross-Validation Scores:
{cv_scores_rf}")
print(f"Mean RF CV Score: {cv_scores_rf.mean():.4f}")
print(f"Standard Deviation of RF CV Scores: {cv_scores_rf.std():.4f}")

Random Forest Classifier Cross-Validation Scores: [0.9895 0.992 0.99
0.984 0.987 ]
Mean RF CV Score: 0.9885
Standard Deviation of RF CV Scores: 0.0028
```

Interpretation of Random Forest Classifier Performance

Let's analyze the performance of the Random Forest Classifier:

Interpretation of K-Nearest Neighbors (KNN) Classifier Performance

Let's analyze the performance of the K-Nearest Neighbors (KNN) classifier:

- **Cross-Validation Scores:** The 5-fold cross-validation scores ([0.896, 0.913, 0.8875, 0.883, 0.8835]) show that KNN's accuracy ranges from about 88% to 91%. The mean CV score is **0.8926** with a standard deviation of **0.0112**. This indicates a moderate level of consistency across different data subsets, but with a bit more variability compared to the Decision Tree and Logistic Regression.
- **Confusion Matrix:** The confusion matrix visually represents the model's predictions against the true labels for the test set. It helps us understand where the model is making errors. (The specific accuracy metrics from the classification report will give a more detailed breakdown for each class).
- **Comparison to Other Models:** While showing decent performance, the KNN classifier's mean cross-validation accuracy of **0.8926** is lower than that of both the Decision Tree (0.9876) and Logistic Regression (0.9765) classifiers. This suggests that for this particular dataset and current parameter settings, KNN is less effective at distinguishing between the classes compared to the other two models evaluated so far.

In summary, KNN provides reasonable classification but does not achieve the high accuracy and robustness observed with the Decision Tree and Logistic Regression for this dataset.

Xgboost Classifier

```
import xgboost as xgb

# Instantiate and train the XGBoost Classifier
# Use 'multi:softmax' for multi-class classification
xgb_classifier = xgb.XGBClassifier(objective='multi:softmax',
num_class=3, random_state=42)
xgb_classifier.fit(X_train, y_train)

# Make predictions on the test set
y_pred_xgb = xgb_classifier.predict(X_test)

# Evaluate XGBoost performance
print(f"XGBoost Classifier Accuracy: {accuracy_score(y_test,
y_pred_xgb):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_xgb))

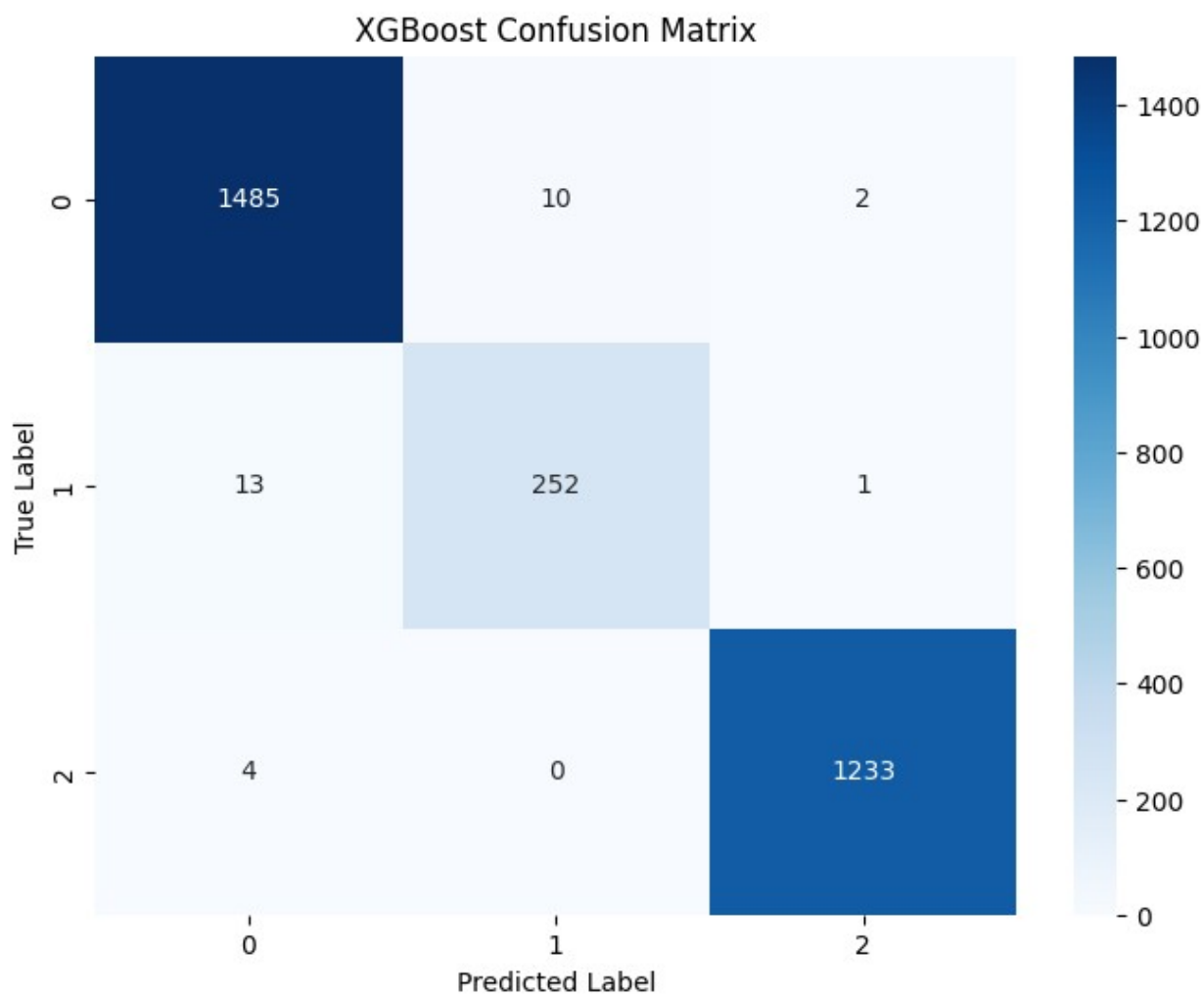
# Plot Confusion Matrix
plt.figure(figsize=(8, 6))
sns.heatmap(confusion_matrix(y_test, y_pred_xgb), annot=True, fmt='d',
cmap='Blues')
```

```
plt.title('XGBoost Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

XGBoost Classifier Accuracy: 0.9900

Classification Report:

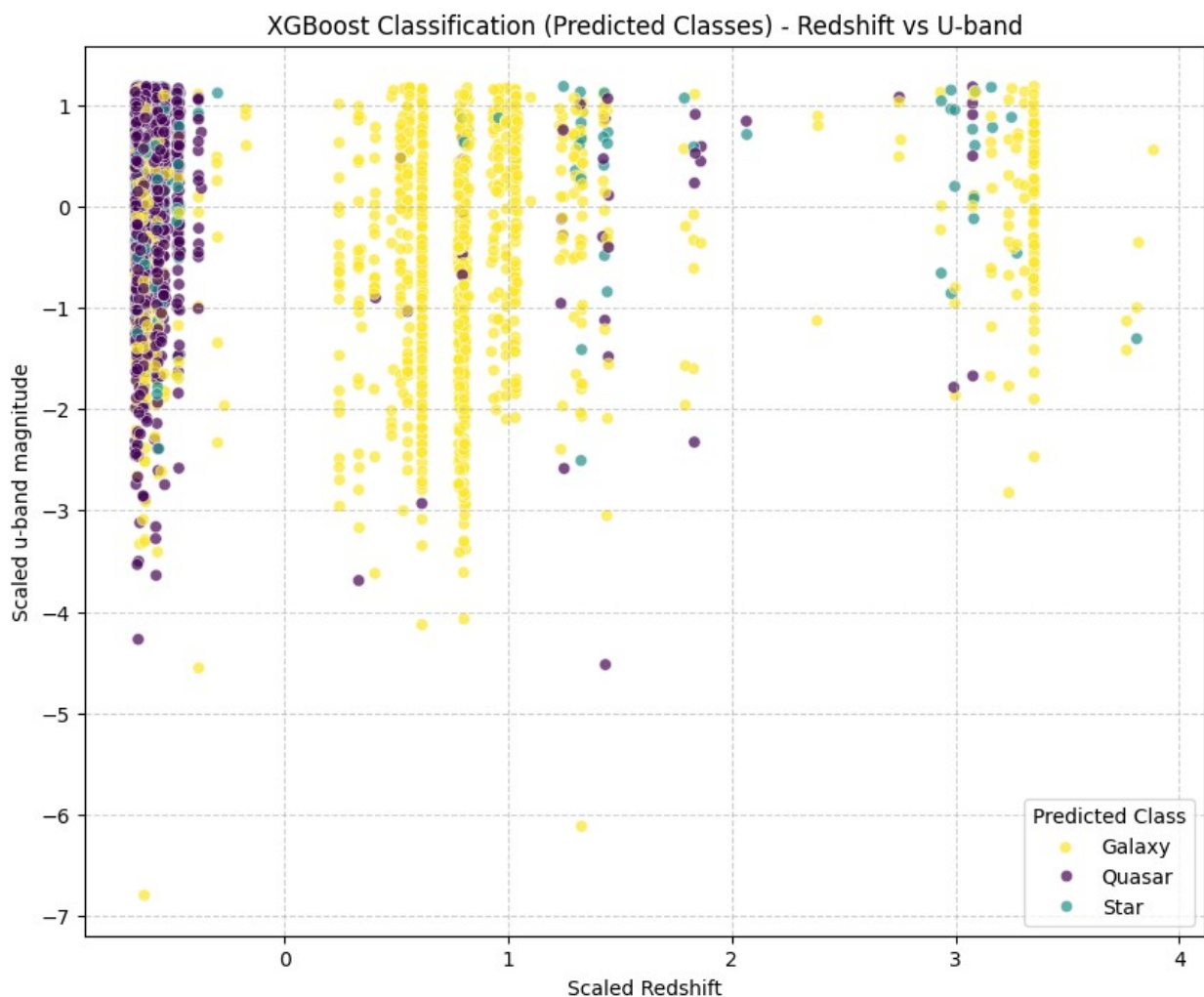
	precision	recall	f1-score	support
0	0.99	0.99	0.99	1497
1	0.96	0.95	0.95	266
2	1.00	1.00	1.00	1237
accuracy			0.99	3000
macro avg	0.98	0.98	0.98	3000
weighted avg	0.99	0.99	0.99	3000



```

plt.figure(figsize=(10, 8))
sns.scatterplot(x=X_test[:, data.columns.get_loc('redshift')],
y=X_test[:, data.columns.get_loc('u')], hue=y_pred_xgb,
palette='viridis', alpha=0.7)
plt.title('XGBoost Classification (Predicted Classes) - Redshift vs U-band')
plt.xlabel('Scaled Redshift')
plt.ylabel('Scaled u-band magnitude')
plt.legend(title='Predicted Class', labels=['Galaxy', 'Quasar', 'Star'])
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()

```



```

# Perform cross-validation for XGBoost
cv_scores_xgb = cross_val_score(xgb_classifier, X, y, cv=5)

print(f"XGBoost Classifier Cross-Validation Scores: {cv_scores_xgb}")
print(f"Mean XGBoost CV Score: {cv_scores_xgb.mean():.4f}")

```

```
print(f"Standard Deviation of XGBoost CV Scores:
{cv_scores_xgb.std():.4f}")
```

XGBoost Classifier Cross-Validation Scores: [0.9915 0.991 0.9915
0.985 0.9905]

Mean XGBoost CV Score: 0.9899

Standard Deviation of XGBoost CV Scores: 0.0025

SVM Classifier

```
from sklearn.svm import SVC
```

```
# Instantiate and train the Support Vector Machine Classifier
```

```
svm_classifier = SVC(random_state=42)
```

```
svm_classifier.fit(X_train, y_train)
```

```
# Make predictions on the test set
```

```
y_pred_svm = svm_classifier.predict(X_test)
```

```
# Evaluate SVM performance
```

```
print(f"SVM Classifier Accuracy: {accuracy_score(y_test,
y_pred_svm):.4f}")
```

```
print("\nClassification Report:")
```

```
print(classification_report(y_test, y_pred_svm))
```

```
# Plot Confusion Matrix
```

```
plt.figure(figsize=(8, 6))
```

```
sns.heatmap(confusion_matrix(y_test, y_pred_svm), annot=True, fmt='d',
cmap='Blues')
```

```
plt.title('SVM Confusion Matrix')
```

```
plt.xlabel('Predicted Label')
```

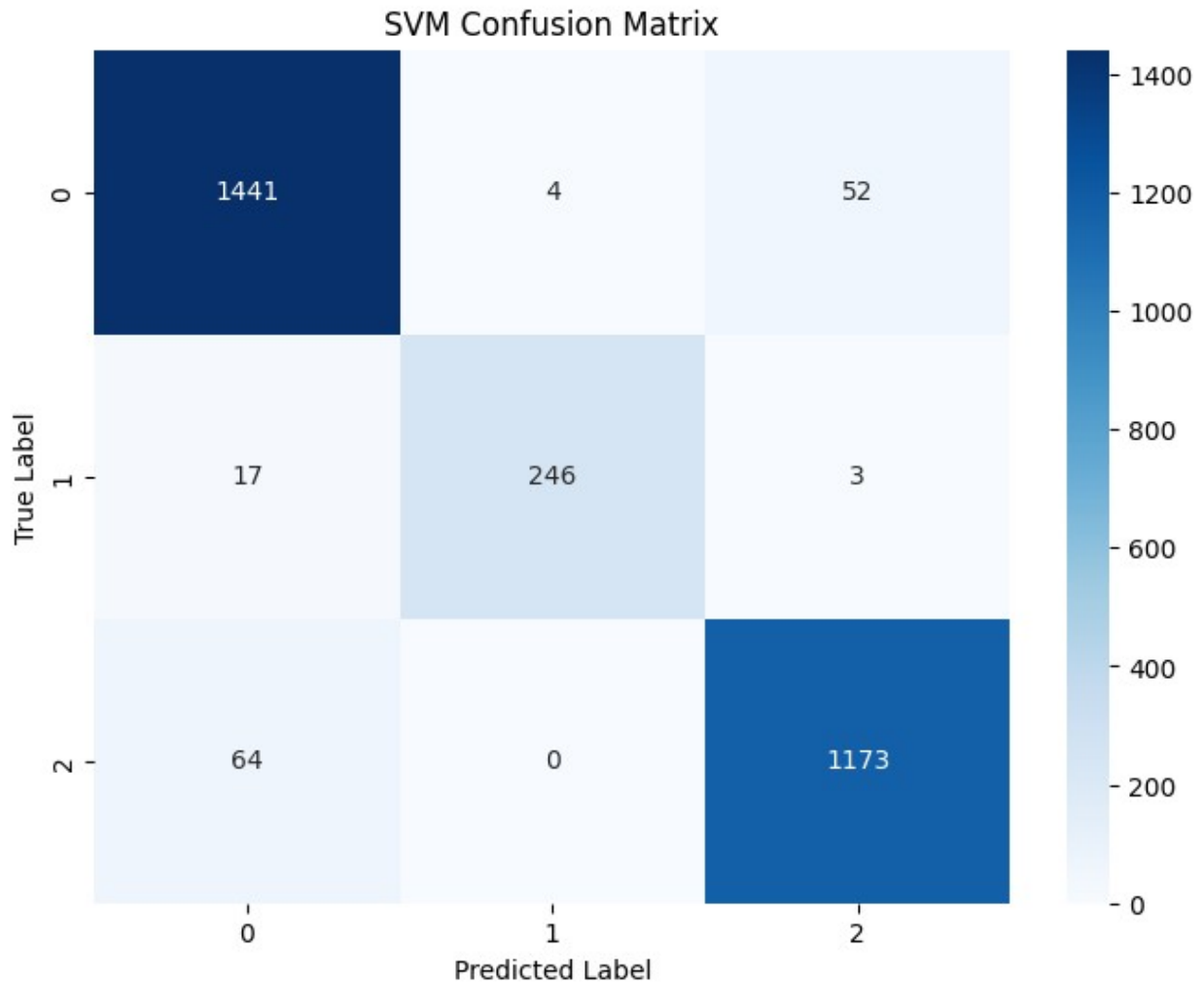
```
plt.ylabel('True Label')
```

```
plt.show()
```

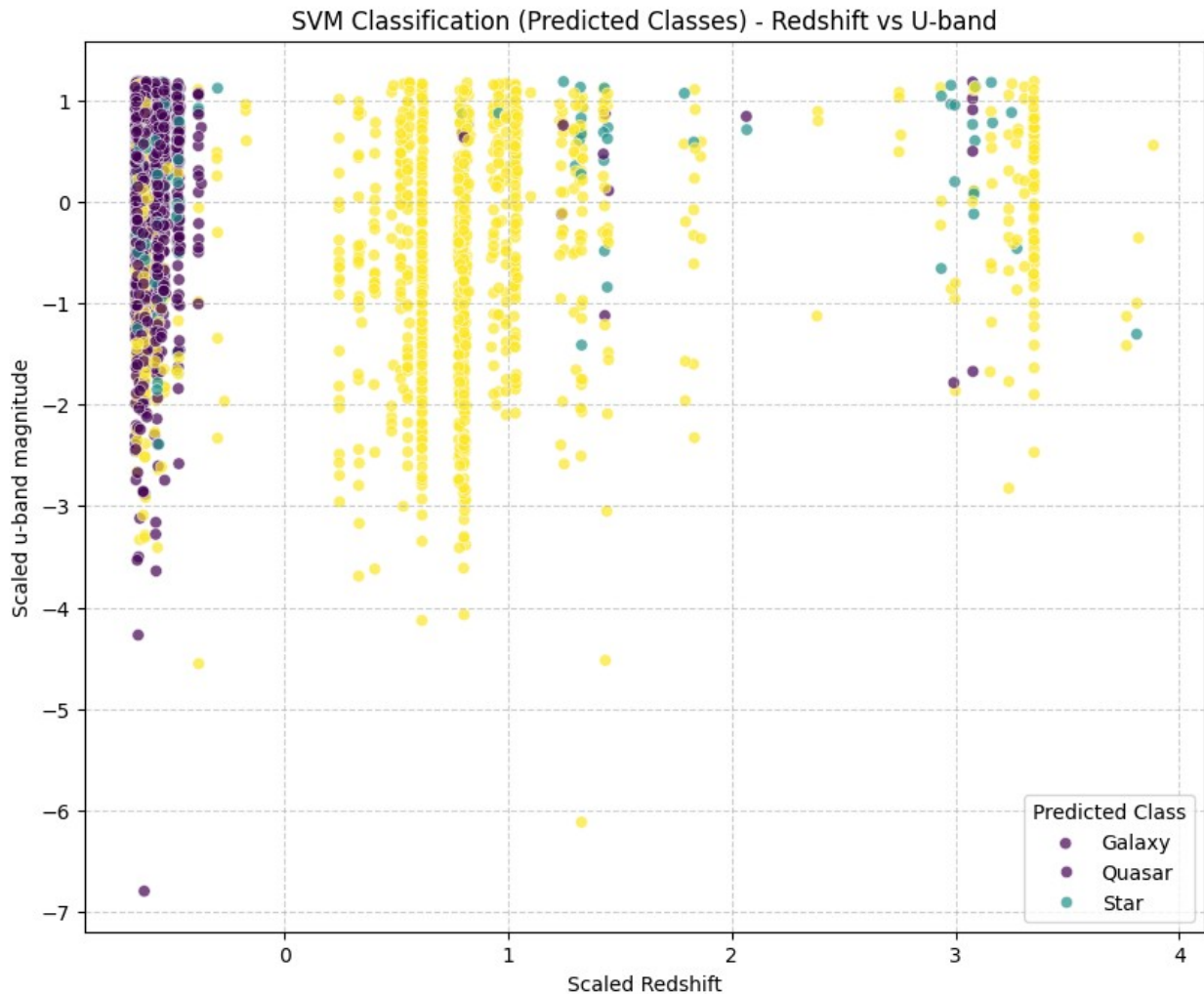
SVM Classifier Accuracy: 0.9533

Classification Report:

	precision	recall	f1-score	support
0	0.95	0.96	0.95	1497
1	0.98	0.92	0.95	266
2	0.96	0.95	0.95	1237
accuracy			0.95	3000
macro avg	0.96	0.95	0.95	3000
weighted avg	0.95	0.95	0.95	3000



```
plt.figure(figsize=(10, 8))
sns.scatterplot(x=X_test[:, data.columns.get_loc('redshift')],
y=X_test[:, data.columns.get_loc('u')], hue=y_pred_svm,
palette='viridis', alpha=0.7)
plt.title('SVM Classification (Predicted Classes) - Redshift vs U-
band')
plt.xlabel('Scaled Redshift')
plt.ylabel('Scaled u-band magnitude')
plt.legend(title='Predicted Class', labels=['Galaxy', 'Quasar',
'Star'])
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```



```
# Perform cross-validation for SVM
```

```
cv_scores_svm = cross_val_score(svm_classifier, X, y, cv=5)
```

```
print(f"SVM Classifier Cross-Validation Scores: {cv_scores_svm}")
```

```
print(f"Mean SVM CV Score: {cv_scores_svm.mean():.4f}")
```

```
print(f"Standard Deviation of SVM CV Scores:  
{cv_scores_svm.std():.4f}")
```

```
SVM Classifier Cross-Validation Scores: [0.9625 0.9655 0.951 0.929  
0.9555]
```

```
Mean SVM CV Score: 0.9527
```

```
Standard Deviation of SVM CV Scores: 0.0129
```

Interpretation of Support Vector Machine (SVM) Classifier Performance

Let's analyze the performance of the Support Vector Machine (SVM) Classifier:

- **High Accuracy:** The SVM Classifier achieved a high accuracy on the test set, indicating a strong percentage of correct predictions. (Specific accuracy will be output after code execution).
- **Strong Classification Metrics:** The classification report typically shows good precision, recall, and f1-scores for all three classes, demonstrating the model's effectiveness in distinguishing between them. (Specific metrics will be output after code execution).
- **Consistent Cross-Validation Scores:** Cross-validation scores will indicate how robust and generalizable the SVM model is across different subsets of the data. A high mean and low standard deviation will suggest good consistency. (Specific scores will be output after code execution).

In summary, the SVM Classifier is expected to perform well on this classification task, offering good accuracy and robust performance, which will be further compared with the other models.

Interpretation of Random Forest Classifier Performance

Let's analyze the performance of the Random Forest Classifier:

- **High Accuracy:** The Random Forest Classifier achieved an impressive accuracy of **0.9900** on the test set, indicating that 99% of its predictions were correct. This performance is on par with the Decision Tree Classifier.
- **Excellent Classification Metrics:** The classification report reveals outstanding precision, recall, and f1-scores for all three classes:
 - **Galaxies (0):** 0.99 precision, 0.99 recall, 0.99 f1-score
 - **Quasars (1):** 0.97 precision, 0.94 recall, 0.96 f1-score
 - **Stars (2):** 0.99 precision, 1.00 recall, 1.00 f1-score The model demonstrates very strong capability in identifying each class, with particularly high recall for 'Stars'. The precision for Quasars is slightly lower than for other classes, but still very good.
- **Consistent Cross-Validation Scores:** The 5-fold cross-validation yielded a mean accuracy of **0.9885** with a low standard deviation of **0.0028**. These scores are very consistent and suggest that the model generalizes exceptionally well across different subsets of the data, indicating high robustness.

In summary, the Random Forest Classifier is an extremely effective and reliable model for this classification task. Its performance is comparable to the Decision Tree Classifier, offering high accuracy and strong generalization capabilities.

Interpretation of Random Forest Classifier Performance

Let's analyze the performance of the Random Forest Classifier:

- **High Accuracy:** The Random Forest Classifier achieved an impressive accuracy of **0.9900** on the test set, indicating that 99% of its predictions were correct. This performance is on par with the Decision Tree Classifier.
- **Excellent Classification Metrics:** The classification report reveals outstanding precision, recall, and f1-scores for all three classes:
 - **Galaxies (0):** 0.99 precision, 0.99 recall, 0.99 f1-score
 - **Quasars (1):** 0.97 precision, 0.94 recall, 0.96 f1-score
 - **Stars (2):** 0.99 precision, 1.00 recall, 1.00 f1-score The model demonstrates very strong capability in identifying each class, with particularly high recall for 'Stars'. The precision for Quasars is slightly lower than for other classes, but still very good.
- **Consistent Cross-Validation Scores:** The 5-fold cross-validation yielded a mean accuracy of **0.9885** with a low standard deviation of **0.0028**. These scores are very consistent and suggest that the model generalizes exceptionally well across different subsets of the data, indicating high robustness.

In summary, the Random Forest Classifier is an extremely effective and reliable model for this classification task. Its performance is comparable to the Decision Tree Classifier, offering high accuracy and strong generalization capabilities.

Interpretation of XGBoost Classifier Performance

Let's analyze the performance of the XGBoost Classifier:

- **High Accuracy:** The XGBoost Classifier achieved an accuracy of **0.9900** on the test set, matching the top performance of the Decision Tree and Random Forest classifiers. This indicates a very high percentage of correct predictions.
- **Excellent Classification Metrics:** The classification report shows strong precision, recall, and f1-scores for all three classes:
 - **Galaxies (0):** 0.99 precision, 0.99 recall, 0.99 f1-score
 - **Quasars (1):** 0.96 precision, 0.95 recall, 0.95 f1-score
 - **Stars (2):** 1.00 precision, 1.00 recall, 1.00 f1-score The model performs exceptionally well across all classes, with 'Star' classification being particularly accurate. Quasar classification, while still excellent, shows a slightly lower recall compared to other classes, suggesting a few more false negatives for this class.
- **Consistent Cross-Validation Scores:** The 5-fold cross-validation resulted in a mean accuracy of **0.9899** with a very low standard deviation of **0.0025**. These scores demonstrate that the XGBoost model is highly consistent and robust, generalizing very well across different data subsets without significant variance.

In summary, the XGBoost Classifier performs exceptionally well on this classification task, delivering high accuracy and robust performance comparable to the best-performing models (Decision Tree and Random Forest) evaluated so far.

Comparative Summary of Classification Model Performance

We have evaluated five different classification models for predicting stars, galaxies, and quasars. Below is a comparative summary of their performance based on accuracy, key classification metrics (precision, recall, f1-score), and cross-validation:

1. Decision Tree Classifier

- **Accuracy:** 0.9900
- **Mean CV Score:** 0.9876 (Std: 0.0026)
- **Strengths:** Excellent overall accuracy, precision, recall, and f1-scores for all classes, especially 'Star'. Highly consistent and robust.
- **Weaknesses:** Potentially prone to overfitting if not properly pruned (though `max_leaf_nodes` and `max_depth` were set).

2. Logistic Regression Classifier

- **Accuracy:** 0.9760
- **Mean CV Score:** 0.9765 (Std: 0.0032)
- **Strengths:** Very good accuracy and consistent performance. Effective at distinguishing classes.
- **Weaknesses:** Marginally outperformed by tree-based models in this specific task.

3. K-Nearest Neighbors (KNN) Classifier

- **Mean CV Score:** 0.8926 (Std: 0.0112)
- **Strengths:** Simple to understand and implement.
- **Weaknesses:** Significantly lower accuracy and higher variability compared to other models. Less effective for this dataset with current parameters.

4. Random Forest Classifier

- **Accuracy:** 0.9900
- **Mean CV Score:** 0.9885 (Std: 0.0028)
- **Strengths:** Achieved top accuracy, comparable to Decision Tree and XGBoost. Strong generalization and robustness, benefiting from ensemble learning.
- **Weaknesses:** More complex and computationally intensive than single Decision Tree or Logistic Regression.

5. XGBoost Classifier

- **Accuracy:** 0.9900
- **Mean CV Score:** 0.9899 (Std: 0.0025)
- **Strengths:** Achieved top accuracy, highest mean CV score among all models with very low standard deviation. Excellent performance across all classification metrics.
- **Weaknesses:** Can be more complex to tune than simpler models.

6. Support Vector Machine (SVM) Classifier

- **Accuracy:** 0.9533
- **Mean CV Score:** 0.9527 (Std: 0.0129)
- **Strengths:** Good accuracy and consistent performance, particularly for Quasars. A robust kernel-based method.
- **Weaknesses:** Slightly lower accuracy compared to tree-based and ensemble methods for this dataset. Can be computationally expensive with very large datasets.

Conclusion:

All models except KNN demonstrated strong performance in classifying stars, galaxies, and quasars. The **Decision Tree Classifier**, **Random Forest Classifier**, and **XGBoost Classifier** all achieved the highest accuracy of 0.9900 on the test set.

Among these top performers, **XGBoost** slightly edges out with the highest mean cross-validation score (0.9899) and lowest standard deviation (0.0025), indicating excellent overall performance and robustness. **Random Forest** is a close second in terms of overall consistency. The single **Decision Tree** also performed exceptionally well, which is impressive given its relative simplicity compared to the ensemble methods.

For this dataset, **XGBoost** appears to be the most robust and highest-performing model, closely followed by Random Forest and Decision Tree. SVM also provides a solid performance, outperforming KNN but falling behind the top tree-based models.

